

**Department of Mathematics and Computer Science**  
**Digital Transformation and Artificial Intelligence 2**

# Implementation of a Private Cloud using OpenStack Report

**Authored by :**

LAKHYARI Aissam  
NIHARMINE Massine  
EL BAHLOULI Halima

**Supervised by :**

ROUTAIB HAYAT

---

# Table of content

---

## Table des matières

|                                                                     |    |
|---------------------------------------------------------------------|----|
| Table of content .....                                              | 2  |
| introduction.....                                                   | 3  |
| I. Cloud.....                                                       | 4  |
| 1. Introduction to Cloud Computing .....                            | 4  |
| 2. History of Cloud Computing .....                                 | 4  |
| 3. Types of Cloud Computing.....                                    | 4  |
| Cloud deployments can be categorized into three main types.....     | 4  |
| 4. Cloud Computing Services .....                                   | 5  |
| 5. Conclusion .....                                                 | 6  |
| II. OpenStack: A Modular Framework for IaaS .....                   |    |
| 1. OpenStack Vs AWS VS OpenNebula .....                             |    |
| III. Architecture OpenStack .....                                   | 12 |
| IV. Create VM .....                                                 | 18 |
| 1. Requirements .....                                               | 18 |
| 2. Creation and Configuration of a Virtual Machine: .....           | 18 |
| 3. Configure Network (Multi Adapters [1-Nat, 2-host-only]).....     | 19 |
| V. Install OpenStack .....                                          | 22 |
| 1. Creating a Non-Root User.....                                    | 22 |
| 2. Cloning the DevStack Repository .....                            | 23 |
| 3. Configuring the ``File`` .....                                   | 23 |
| 4. Configures the environment of OpenStack .....                    | 29 |
| 5. Key Differences Between admin-openrc.sh and demo-openrc.sh ..... | 30 |
| VI. Creation of instances .....                                     | 35 |
| 1. Creation of an Ubuntu instance: .....                            | 35 |
| 2. Creation of a New Project: .....                                 | 37 |
| 3. Configure Network.....                                           | 40 |
| 1. Create an Instance .....                                         | 48 |
| The set of rules created.....                                       | 54 |
| VII. Challenges encountered .....                                   | 63 |
| 1. Network Configuration: .....                                     | 66 |
| Resource Management: .....                                          | 67 |
| Disk Performance:.....                                              | 67 |
| Conclusion.....                                                     | 68 |
| Bibliography.....                                                   | 69 |

# Introduction

---

OpenStack is a leading open-source platform for building private clouds, offering organizations the flexibility to manage compute, storage, and networking resources. By enabling businesses to create customizable and secure cloud environments, OpenStack has become a preferred solution for organizations that prioritize data control, regulatory compliance, and operational efficiency over the limitations of public cloud offerings.

The deployment of OpenStack follows a systematic approach. It begins with preparing the physical or virtual infrastructure and installing a compatible Linux operating system. Networking is configured to enable seamless communication between nodes(can be servers, each server contains a stack principal components), followed by the installation of OpenStack's core components, including Keystone for identity management, Nova for compute resources, Neutron for networking, and Cinder for storage. Setting up the controller node, deploying compute nodes, and integrating storage and networking are crucial to ensuring a functional environment.

Once the deployment is complete, testing is conducted to verify system functionality, while tools like the Horizon dashboard provide a user-friendly interface for cloud management. Additional security measures, such as firewalls and access controls, combined with monitoring and maintenance tools, ensure the long-term stability and security of the private cloud.

This report outlines the key steps involved in deploying a private OpenStack cloud, providing insights into best practices and challenges to help organizations successfully leverage this powerful platform.

# I. Cloud

---

## **1. Introduction to Cloud Computing**

Cloud computing has revolutionized the way organizations store, access, and manage data and applications. It refers to the delivery of computing resources—such as servers, storage, databases, networking, software, and analytics—over the internet, eliminating the need for on-premises infrastructure. This technology enables on-demand access, scalability, flexibility, and cost-efficiency, making it a cornerstone of modern IT operations.

## **2. History of Cloud Computing**

The concept of cloud computing originated in the 1960s, with the idea of utility computing envisioned by computer scientists like John McCarthy. However, its modern form began to take shape in the late 1990s with the advent of virtualization technology and large-scale internet connectivity. Companies like Salesforce, Amazon Web Services (AWS), and Google pioneered the commercial adoption of cloud services in the early 2000s. Since then, cloud computing has become an essential part of digital transformation worldwide.

## **3. Types of Cloud Computing**

**Cloud deployments can be categorized into three main types:**

- **Private Cloud:** A private cloud is dedicated to a single organization, offering enhanced security, control, and customization. It is hosted either on-premises or in a privately managed data center. This type of cloud is ideal for organizations with strict compliance and data sovereignty requirements.

*Example:* Banks or government institutions using private clouds to ensure sensitive data is protected.

- **Public Cloud:** A public cloud is owned and operated by third-party providers like AWS, Microsoft Azure, or Google Cloud, offering shared resources to multiple users over the internet. Public clouds are scalable and cost-efficient, suitable for businesses seeking to reduce capital expenditure.

*Example:* Startups using public clouds to quickly deploy applications without upfront infrastructure costs.

- **Hybrid Cloud:** A hybrid cloud combines private and public cloud environments, enabling data and application sharing between them. This approach offers the flexibility to scale resources based on demand while maintaining critical workloads in a secure private cloud.

*Example:* Retail companies managing customer-sensitive data in a private cloud and hosting their e-commerce websites on a public cloud

## **4. Cloud Computing Services**

Cloud computing services are broadly categorized into three main models, each catering to different business needs:

- ◆ **Infrastructure as a Service (IaaS):**  
IaaS provides virtualized computing resources, such as servers, storage, and networking, over the internet. It allows businesses to scale infrastructure on demand without owning physical hardware.

*Examples:* AWS EC2, Google Compute Engine, Microsoft Azure Virtual Machines.

*Use Case:* Hosting applications, disaster recovery, or running virtual machines.

- ◆ **Platform as a Service (PaaS):**  
PaaS offers a platform for developers to build, test, and deploy applications without managing the underlying infrastructure. It simplifies the development process and reduces the time-to-market.

*Examples:* Google App Engine, AWS Elastic Beanstalk, Microsoft Azure App Service.

*Use Case:* Building web and mobile applications.

◆ **Software as a Service (SaaS):**

SaaS delivers software applications over the internet on a subscription basis. Users can access these applications via a web browser without worrying about installation, maintenance, or updates.

*Examples:* Google Workspace (formerly G Suite), Salesforce, Microsoft 365.

*Use Case:* Email, customer relationship management (CRM), and collaboration tools.

## **5. Conclusion**

Cloud computing has become a cornerstone of modern IT strategies, offering diverse deployment models and services to meet varying organizational needs. Whether through private clouds for enhanced security, public clouds for scalability, or hybrid clouds for flexibility, businesses can harness the power of the cloud to achieve operational efficiency and innovation. Understanding the various cloud services—such as IaaS, PaaS, and SaaS—enables organizations to choose solutions that align with their strategic goals, ensuring long-term success in a digital-first world.

# II. OpenStack: A Modular Framework for IaaS

---

OpenStack is an open-source platform designed to deliver Infrastructure as a Service (IaaS), enabling organizations to deploy and manage cloud infrastructure efficiently. It comprises a collection of modular services that work together seamlessly to provide compute, storage, and networking resources. These services are highly flexible and can be deployed individually or in combination, depending on the organization's specific requirements. Below is an overview of its core components:

✧ **Keystone (Identity Service)**

Keystone acts as the central authentication and authorization service in OpenStack. It provides identity management, token-based access control, service cataloging, and policy enforcement. This ensures that only authenticated users and applications can interact with OpenStack resources, enhancing security.

*Example:* Managing user access to virtual machines or storage resources.

✧ **Nova (Compute Service)**

Nova is the compute engine of OpenStack, responsible for provisioning and managing virtual machines (VMs) and compute resources. It supports multiple hypervisors, such as KVM, VMware, and Hyper-V, making it versatile for various infrastructure setups.

*Example:* Launching virtual servers for hosting applications or workloads.

✧ **Neutron (Networking Service)**

Neutron provides "Networking as a Service" (NaaS) capabilities, allowing users to create and manage complex virtual networks. It supports VLANs, software-defined networking (SDN), and network function virtualization (NFV), enabling advanced network configurations like load balancing and firewalls.

*Example:* Setting up isolated networks for multi-tenant environments.

✧ **Glance (Image Service)**

Glance handles the storage, discovery, and management of disk images. These images can be used to launch virtual machines, ensuring consistency and rapid deployment. It supports various formats such as RAW, VHD, VMDK, and QCOW2.

*Example:* Hosting pre-configured images for operating systems and application stacks.

✧ **Cinder (Block Storage Service)**

Cinder provides persistent block storage for virtual machines. It enables the creation, attachment, and management of block storage volumes, which can be resized and retained even after VM termination.

*Example:* Providing additional storage for database applications or file systems.

✧ **Swift (Object Storage Service)**

Swift offers scalable and redundant object storage for unstructured data such as backups, archives, or media files. It ensures data durability by replicating objects across multiple nodes and data centers.

*Example:* Storing large datasets, videos, or application logs.

✧ **Horizon (Dashboard)**

Horizon is OpenStack's web-based user interface that simplifies cloud management. It allows users to manage services like virtual machines, networks, and storage through an intuitive dashboard without relying on command-line interfaces.

*Example:* Creating and managing instances, networks, and storage volumes via a browser.



✧ **Customizability and Adaptability**

OpenStack's modular design makes it highly customizable, allowing organizations to tailor deployments to meet specific needs, whether for small-scale private clouds or large-scale multi-tenant public clouds. By combining these components, businesses can create robust and scalable cloud environments that align with their operational goals.

## 1. Opensatck Vs AWS VS openNebula

| Key Aspect                | OpenStack                                                      | AWS (Amazon Web Services)                                    | OpenNebula                                                       |
|---------------------------|----------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------|
| Type                      | Open-source cloud platform for IaaS                            | Public cloud service provider                                | Open-source cloud platform for IaaS                              |
| Deployment Model          | Private cloud, public cloud, or hybrid cloud                   | Public cloud (AWS services), supports hybrid via Outposts    | Private cloud, public cloud, or hybrid cloud                     |
| Ownership                 | Community-driven, open-source                                  | Proprietary, operated by Amazon                              | Community-driven, open-source                                    |
| Ease of Setup             | Complex setup requiring technical expertise                    | Easy to start with pre-built services                        | Easier to set up than OpenStack but still requires configuration |
| Target Users              | Enterprises, service providers, developers with control needs  | Businesses of all sizes needing scalable, ready-to-use cloud | Small to medium businesses, research institutions                |
| Scalability               | Highly scalable but depends on infrastructure                  | Extremely scalable with global data centers                  | Scalable but primarily suited for smaller deployments            |
| Cost                      | Free to use (open-source), with operational costs for hardware | Pay-as-you-go pricing for resources and services             | Free to use (open-source), with operational costs for hardware   |
| Ecosystem and Integration | Large ecosystem with support for third-party tools             | Comprehensive ecosystem with integrated services             | Smaller ecosystem compared to OpenStack and AWS                  |

| Key Aspect                | OpenStack                                                         | AWS (Amazon Web Services)                                     | OpenNebula                                                            |
|---------------------------|-------------------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------|
| Compute Services          | Nova for VM management and compute provisioning                   | Amazon EC2, Lambda for serverless computing                   | Supports VM and container management via integrated components        |
| Storage Options           | Cinder (block storage), Swift (object storage)                    | S3 (object storage), EBS (block storage), Glacier (archival)  | Block, file, and object storage, but more limited than OpenStack      |
| Networking                | Neutron for software-defined networking and VLAN management       | Amazon VPC for virtual networks                               | Native support for VLANs and simplified networking                    |
| Orchestration             | Heat for resource orchestration using templates                   | AWS CloudFormation for infrastructure templates               | Native orchestration with basic features                              |
| Monitoring                | Ceilometer for metering and monitoring                            | AWS CloudWatch for detailed monitoring and alerts             | Basic monitoring tools, less advanced than OpenStack or AWS           |
| Security                  | Keystone for identity and access management                       | Integrated IAM (Identity and Access Management)               | Basic identity management, less feature-rich than others              |
| Support and Documentation | Extensive community support, with commercial options available    | Professional, paid support with extensive documentation       | Smaller community but offers commercial support options               |
| Global Reach              | Relies on private infrastructure or hosting provider locations    | Global presence with multiple regions and availability zones  | Relies on private infrastructure or hosting provider locations        |
| Use Cases                 | Enterprises needing complete control over their cloud environment | Businesses requiring scalable and ready-to-use cloud services | Organizations seeking a lightweight and cost-effective cloud solution |

**OpenStack:** Best for organizations requiring a highly customizable private cloud solution with complete control over infrastructure. Ideal for large-scale deployments.

**AWS:** Suited for businesses seeking an easy-to-use, globally available public cloud with comprehensive services. Requires no upfront investment in infrastructure.

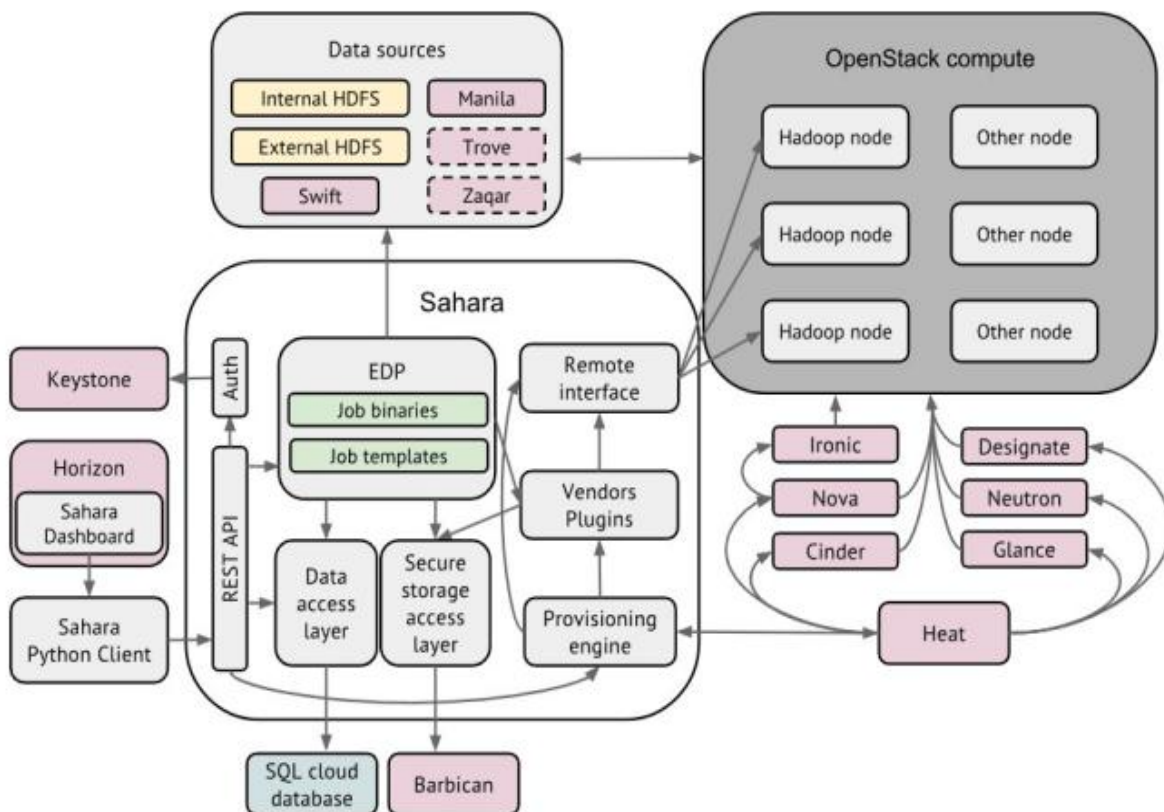
**OpenNebula:** A simpler alternative to OpenStack, focused on small to medium businesses needing lightweight private cloud solutions.

**AWS, OpenStack, and OpenNebula** are prominent cloud computing platforms, each with distinct use cases. **AWS** excels in scalability, reliability, and a broad range of services, making it ideal for enterprises requiring extensive resources, global reach, and advanced tools for artificial intelligence and machine learning. **OpenStack** is an open-source platform designed for organizations seeking customizable private or hybrid cloud solutions, with a focus on flexibility and control. **OpenNebula**, another open-source platform, is simpler to deploy and manage, catering to smaller-scale private clouds or edge computing use cases. Choosing the right platform depends on factors such as the organization's scale, budget, technical expertise, and specific goals.

For a digital transformation and artificial intelligence engineer, AWS is often the best choice due to its advanced AI services and robust ecosystem, though OpenStack may be preferred if the focus is on building highly tailored cloud environments.

### III. Architecture OpenStack

---



This diagram illustrates the integration of various OpenStack components working cohesively to manage and deploy big data processing frameworks, such as Hadoop, using Sahara (the OpenStack Data Processing service). At its core, Sahara serves as a provisioning and management layer, enabling the deployment of data processing clusters on OpenStack compute resources.

It interacts with Keystone for authentication, Horizon for dashboard integration, and Heat for orchestration. Data sources such as Swift (object storage), Manila (file storage),

and HDFS (both internal and external) are utilized for data input and storage. Additional services, including Nova for compute resource management, Cinder for block storage, Neutron for networking, and Glance for image management, play integral roles in provisioning and supporting the infrastructure. Barbican ensures secure storage of sensitive information, while Sahara's plugins and job templates streamline cluster configuration.

Together, these components demonstrate OpenStack's modular architecture and its ability to integrate seamlessly for specialized tasks like big data processing.

---

## Sahara

**Role:** Sahara is OpenStack's Data Processing service, designed to provision and manage big data frameworks like Hadoop, Spark, and Storm.

### How It Works:

Sahara allows users to create and manage data processing clusters with minimal effort.

It integrates with other OpenStack components to orchestrate and provision clusters dynamically.

Key features include job templates, job binaries (scripts, programs), and plugins for various big data engines.

It uses the REST API for communication and enables management via the Sahara Dashboard within Horizon or through the Sahara Python Client.

---

## Keystone

**Role:** Keystone provides identity, token, catalog, and policy services for OpenStack authentication and authorization.

### How It Works:

Keystone ensures secure access by validating user credentials and providing authentication tokens.

Sahara uses Keystone for managing user access and secure integration with other OpenStack components.

---

## **Horizon (Sahara Dashboard)**

Role: {key Manager switch} Horizon is the web-based dashboard interface for OpenStack, with Sahara integrated for managing data processing tasks.

How It Works:

It provides a graphical user interface for users to configure, deploy, and monitor Sahara clusters.

This reduces the need for complex command-line interactions, making big data provisioning accessible.

---

## **Heat**

Role: Heat is OpenStack's orchestration service, which automates infrastructure deployment through templates.

How It Works:

Heat coordinates the creation of the virtual machines, networking, and storage resources required for Sahara clusters.

Sahara leverages Heat templates to define and deploy big data clusters efficiently.

---

## **OpenStack Compute**

Role: Manages compute resources using Nova for provisioning and maintaining virtual machines.

How It Works:

Sahara provisions clusters by launching VMs (nodes) using Nova.

The compute nodes are configured to run as either Hadoop nodes or other roles (e.g., Spark master/slave).

---

## Data Sources

### Swift:

Provides object storage for storing unstructured data like logs, input files, or outputs from big data jobs.

Sahara integrates with Swift to use it as a data repository for processing tasks.

### Manila:

Offers shared file system storage, enabling data sharing across multiple nodes.

Ideal for cases where applications in the cluster require shared access to data files.

---

### External/Internal HDFS:

Hadoop Distributed File System (HDFS) acts as the primary storage layer for Hadoop clusters.

Sahara can work with both external HDFS (pre-existing setups) or configure internal HDFS storage.

### Trove and Zaqr:

Trove provides database as a service (DBaaS), while Zaqr offers messaging as a service for coordination in distributed systems.

---

## Plugins and Job Management

*Role:* Sahara's Vendors Plugins enable integration with specific big data technologies like Hadoop or Spark.

### *How It Works:*

Plugins provide configurations for deploying clusters tailored to a specific framework.

Job templates define how data processing tasks (like MapReduce jobs) are executed across the cluster.

EDP (Elastic Data Processing) simplifies running data processing workflows.

---

## **Cinder (Block Storage)**

Role: Provides persistent block storage for instances in the cluster.

How It Works:

Sahara integrates with Cinder to attach block storage volumes to the compute nodes for data-intensive workloads.

---

## **Neutron (Networking)**

Role: Manages networking for communication between cluster nodes and external systems.

How It Works:

Sahara uses Neutron to configure VLANs, SDN, or flat networks for seamless connectivity between nodes.

Ensures that nodes within a cluster can communicate effectively.

---

## **Glance (Image Service)**

Role: Manages the VM images used to provision nodes in the Sahara clusters.

How It Works:

Glance stores pre-configured images for Hadoop or other big data frameworks.

Sahara uses these images to quickly deploy clusters without manual configuration of individual nodes.

---

## **Barbican (Key Management Service)**



Role: Ensures secure storage and management of sensitive information like encryption keys and certificates.

How It Works:

Sahara integrates with Barbican to protect sensitive data and credentials used during cluster provisioning.

---

## **Ceph Integration (Not Explicitly Shown)**

Role: Offers distributed object, block, and file storage that complements other OpenStack storage solutions like Swift and Cinder.

How It Works:

If integrated, Ceph enhances scalability and redundancy for storage needs in Sahara clusters.

---

## **The relationship between these components is as follows:**

Sahara uses Nova to provision and manage the VM instances necessary to create data processing clusters.

Data sources can be integrated into the clusters deployed by Sahara to enable the processing of data from various sources.

Nova provides the underlying infrastructure, while Sahara handles the cluster management layer for data processing.

In summary, Nova provides the computing infrastructure in the form of virtual machines, Sahara uses this infrastructure to create data processing clusters, and data sources are processed by these clusters to perform data processing operations.

# IV. Create VM

---

## **1. Requirements**

You need at least :

- RAM: 8 GB
- CPU: with at least 2 cores
- Storage: 80 GB
- OS type to be installed: Linux
- OS version to be installed: Ubuntu-22.04.5

The first step is to install and configure VirtualBox, which is a type 2 hypervisor that allows multiple virtual operating systems (guests) to run on a single host machine.

**The tools that will be used throughout this lab are as follows:**

- Oracle VM VirtualBox version 7.0.2.
- An ISO file for installing Ubuntu Desktop 22.04.3.

## **2. Creation and Configuration of a Virtual Machine:**

The goal of this first step is to create a new virtual machine using VirtualBox's graphical interface. During this initial step, we will define the basic settings of the virtual machine, including its name, the amount of allocated RAM, and the storage configuration.

- Name: openstack
- RAM: 5 GB
- CPU: 3
- Storage: 80 GB
- OS type to be installed: Linux
- OS version to be installed: Ubuntu-22.04.5

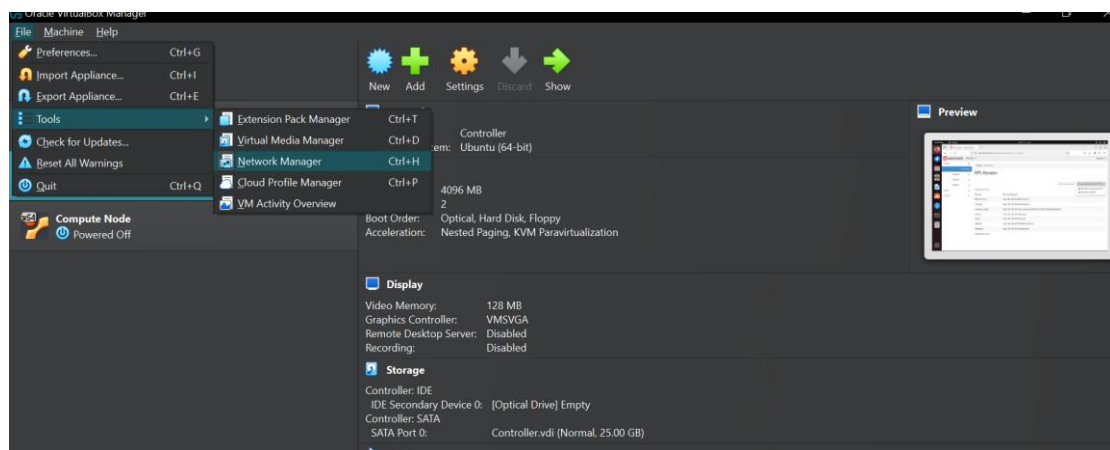
Create a new Vm Machine:

```
sudo apt update && sudo apt upgrade -y && sudo apt dist-upgrade
```

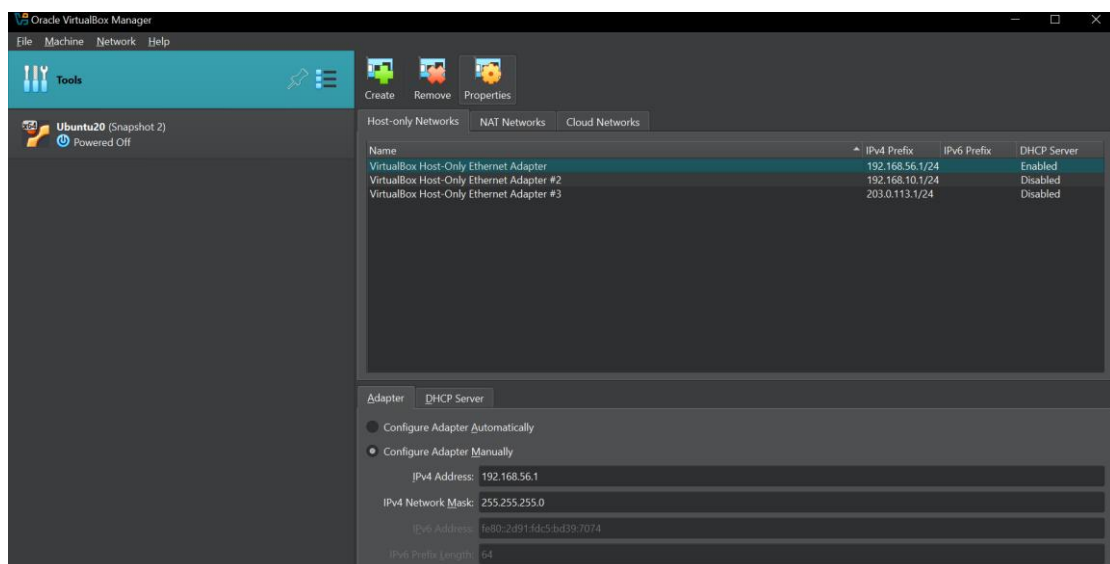
```
sudo apt-get install bridge-utils -y && sudo apt install net-tools -y
```

### 3. Configure Network (Multi Adapters [1-Nat, -2host-only])

Access the Network Manager section:

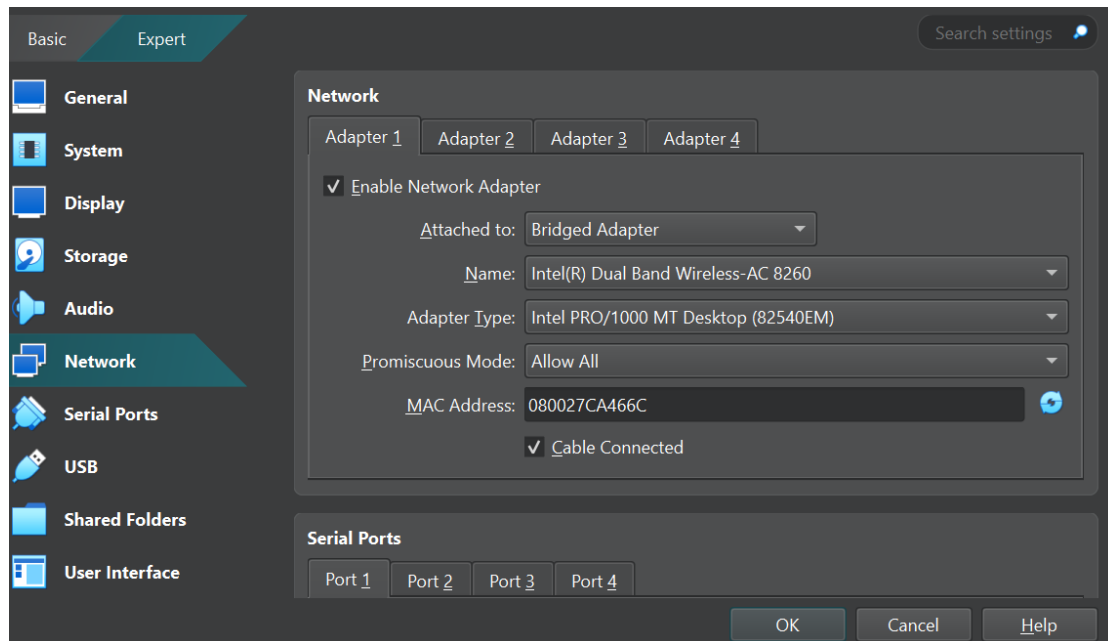


Create a new Network interface and configure it based on your requirements :

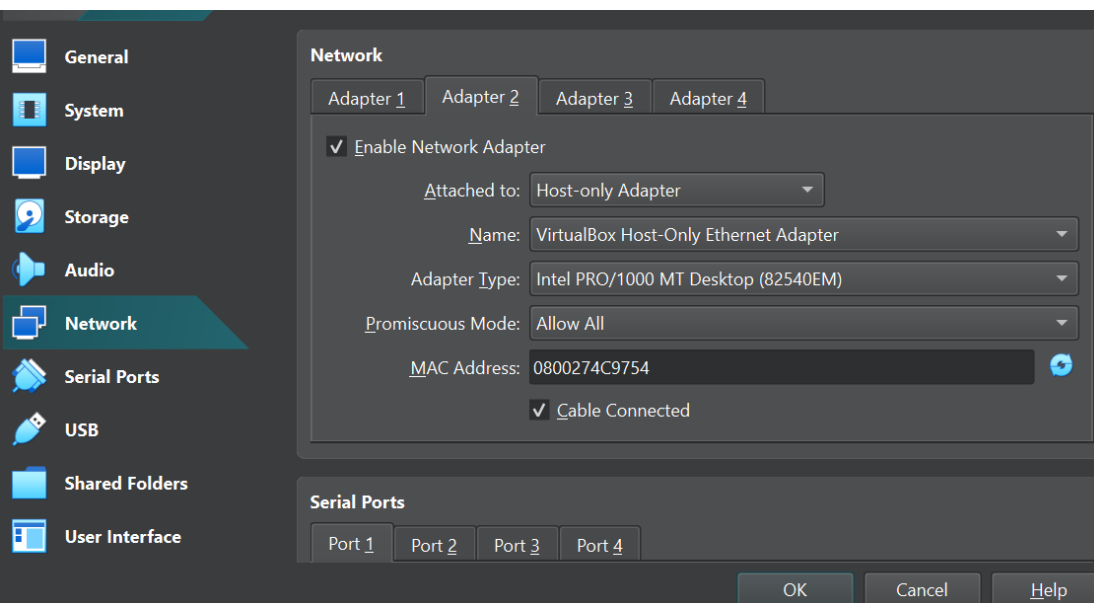


In my case, I created a single network interface ('Virtual Network Adapter') and configured it manually to use the IP address range 192.168.56.1/24.

Ensure that the DHCP is disabled.



First network adapter attached to “Bridge adapter” Mode.



The second adapter is set to 'Host-only Adapter' mode, which will be used by OpenStack's services to communicate with each other without any interaction with the external network.

Access Your Vm machine and then check the network configurations , using the command “ifconfig” :

```

root@massin-VirtualBox: /home/massin/Downloads# ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.140 netmask 255.255.255.0 broadcast 192.168.2.255
    inet6 fe80::25ee:e400:decd:9f64 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:ae:f8:9b txqueuelen 1000 (Ethernet)
    RX packets 6082 bytes 3485030 (3.4 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2853 bytes 472347 (472.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::27d4:1e6d:e07:75d8 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:c0:ec:eb txqueuelen 1000 (Ethernet)
    RX packets 130 bytes 25652 (25.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 186 bytes 25763 (25.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 185626 bytes 44927239 (44.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 185626 bytes 44927239 (44.9 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

In this configuration, there are two main network adapters: **enp0s3**, configured with the IP range 192.168.2.x, and **enp0s8**, configured with the IP range 192.168.56.x. Both adapters are active and assigned unique subnets.

## v. Install OpenStack

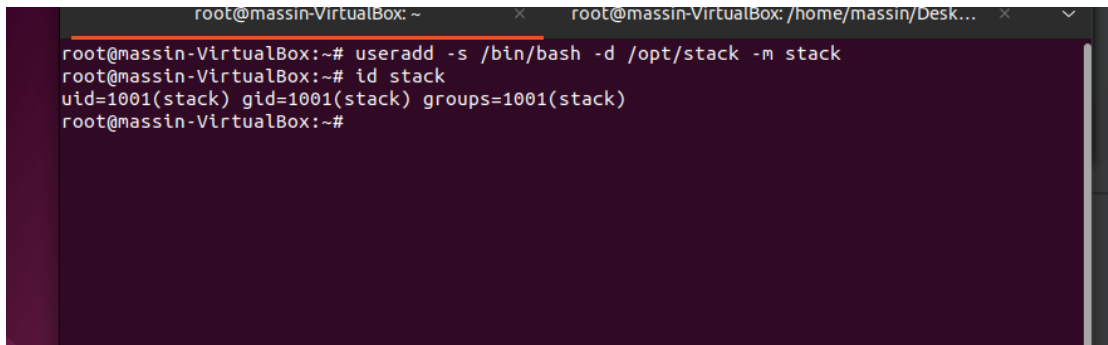
---

### 1. Creating a Non-Root User

DevStack requires a non-root user with sudo privileges.

Create a user:

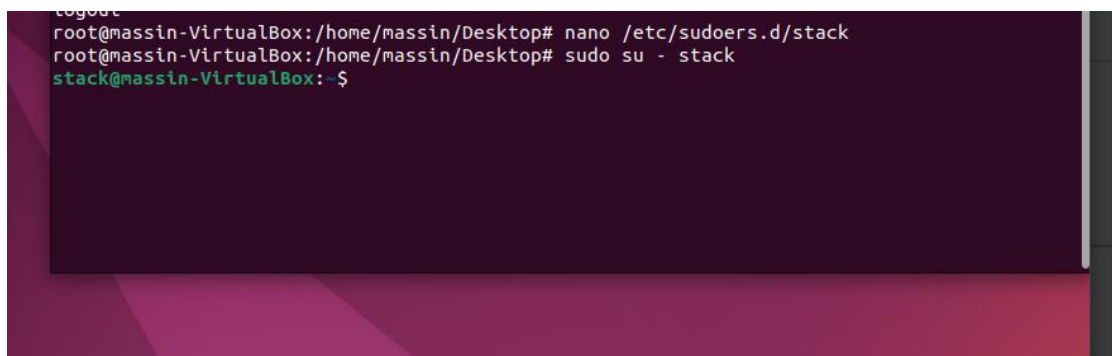
```
sudo useradd -s /bin/bash -d /opt/stack -m stack
echo "stack ALL=(ALL) NOPASSWD:ALL" | sudo tee
/etc/sudoers.d/stack
```

A terminal window with a dark purple background. The prompt is root@massin-VirtualBox:~#. The user runs 'useradd -s /bin/bash -d /opt/stack -m stack'. The prompt changes to root@massin-VirtualBox:~#. The user then runs 'id stack'. The output is 'uid=1001(stack) gid=1001(stack) groups=1001(stack)'. The prompt returns to root@massin-VirtualBox:~#.

```
root@massin-VirtualBox: ~ x root@massin-VirtualBox: /home/massin/Desktop... x
root@massin-VirtualBox:~# useradd -s /bin/bash -d /opt/stack -m stack
root@massin-VirtualBox:~# id stack
uid=1001(stack) gid=1001(stack) groups=1001(stack)
root@massin-VirtualBox:~#
```

Switch to the `stack` user:

```
su - stack
```

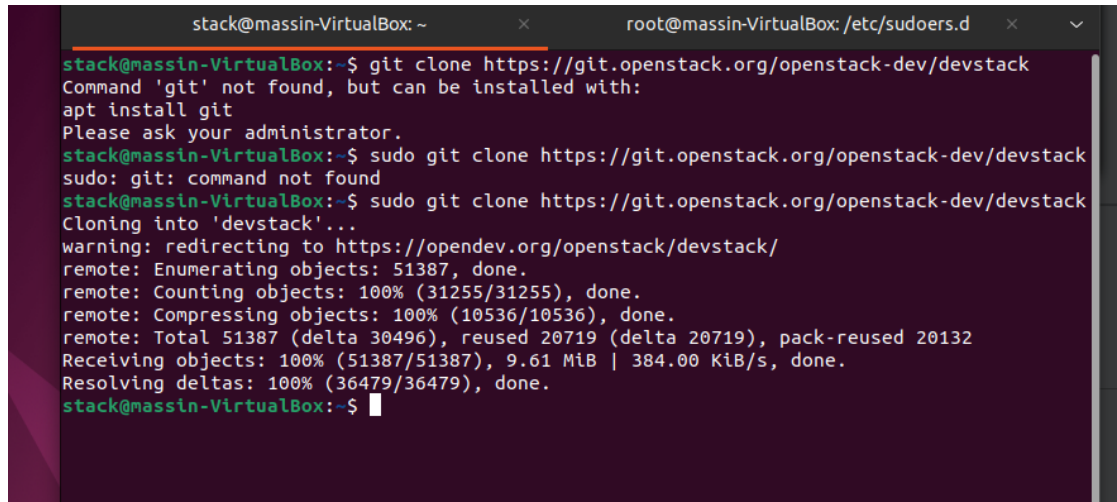
A terminal window with a dark purple background. The prompt is root@massin-VirtualBox:/home/massin/Desktop#. The user runs 'nano /etc/sudoers.d/stack'. The prompt changes to root@massin-VirtualBox:/home/massin/Desktop#. The user then runs 'sudo su - stack'. The prompt changes to stack@massin-VirtualBox:~\$.

```
root@massin-VirtualBox:/home/massin/Desktop# nano /etc/sudoers.d/stack
root@massin-VirtualBox:/home/massin/Desktop# sudo su - stack
stack@massin-VirtualBox:~$
```

## 2. Cloning the DevStack Repository

Clone the official DevStack repository:

```
git clone https://opendev.org/openstack/devstack  
  
cd devstack
```



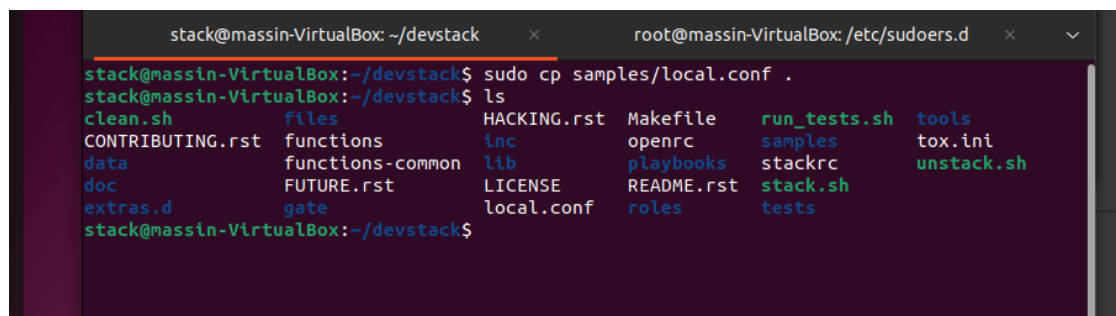
A terminal window showing the process of cloning the DevStack repository. The user is in a shell as 'stack' on a 'massin-VirtualBox'. They attempt to run 'git clone https://git.openstack.org/openstack-dev/devstack', but 'git' is not found. They then run 'sudo git clone https://git.openstack.org/openstack-dev/devstack', which also fails because 'git' is not in the PATH. Finally, they run 'sudo git clone https://opendev.org/openstack/devstack', which succeeds. The output shows the cloning progress, including enumerating objects, counting objects, compressing objects, and receiving objects. The repository is cloned into a directory named 'devstack'.

```
stack@massin-VirtualBox:~$ git clone https://git.openstack.org/openstack-dev/devstack  
Command 'git' not found, but can be installed with:  
apt install git  
Please ask your administrator.  
stack@massin-VirtualBox:~$ sudo git clone https://git.openstack.org/openstack-dev/devstack  
sudo: git: command not found  
stack@massin-VirtualBox:~$ sudo git clone https://opendev.org/openstack/devstack  
Cloning into 'devstack'...  
warning: redirecting to https://opendev.org/openstack/devstack/  
remote: Enumerating objects: 51387, done.  
remote: Counting objects: 100% (31255/31255), done.  
remote: Compressing objects: 100% (10536/10536), done.  
remote: Total 51387 (delta 30496), reused 20719 (delta 20719), pack-reused 20132  
Receiving objects: 100% (51387/51387), 9.61 MiB | 384.00 KiB/s, done.  
Resolving deltas: 100% (36479/36479), done.  
stack@massin-VirtualBox:~$
```

## 3. Configuring the `` File``

The `local.conf` file is used to customize the DevStack setup.

Copy the file:



A terminal window showing the user in the 'devstack' directory. They run 'sudo cp samples/local.conf .' to copy the 'local.conf' file from the 'samples' directory to the current directory. The command is successful. Below the command, a 'ls' command is run, showing the contents of the 'devstack' directory. The output lists various files and directories, including 'clean.sh', 'CONTRIBUTING.rst', 'data', 'doc', 'extras.d', 'files', 'functions', 'functions-common', 'FUTURE.rst', 'gate', 'HACKING.rst', 'inc', 'lib', 'LICENSE', 'local.conf', 'Makefile', 'openrc', 'playbooks', 'README.rst', 'roles', 'run\_tests.sh', 'samples', 'stackrc', 'stack.sh', 'tests', 'tools', 'tox.ini', and 'unstack.sh'.

```
stack@massin-VirtualBox:~/devstack$ sudo cp samples/local.conf .  
stack@massin-VirtualBox:~/devstack$ ls  
clean.sh      files          HACKING.rst   Makefile      run_tests.sh  tools  
CONTRIBUTING.rst  functions     inc           openrc        samples       tox.ini  
data          functions-common  lib          playbooks    stackrc       unstack.sh  
doc           FUTURE.rst     LICENSE       README.rst   stack.sh  
extras.d      gate          local.conf    roles        tests
```

Get Address Ip and name of The network interface :

```
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.200.141 netmask 255.255.255.0 broadcast 192.168.200.255
    inet6 fe80::25ee:e400:decd:9f64 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:ae:f8:9b txqueuelen 1000 (Ethernet)
    RX packets 938699 bytes 1279959522 (1.2 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 172093 bytes 13765938 (13.7 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::27d4:1e6d:e07:75d8 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:c0:ec:eb txqueuelen 1000 (Ethernet)
    RX packets 204 bytes 37130 (37.1 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 215 bytes 32138 (32.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 287882 bytes 119383671 (119.3 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 287882 bytes 119383671 (119.3 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

virbr0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 192.168.122.1 netmask 255.255.255.0 broadcast 192.168.122.255
    ether 52:54:00:42:47:ec txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Modify the file **local.conf** :

The local.conf file is a configuration file for DevStack. It allows the user to specify various configuration parameters such as which OpenStack services to enable, IP addresses, passwords, etc. It plays a crucial role in customizing the OpenStack installation via DevStack.

So, we make some modifications to the local.conf file to customize the settings:

=> Set passwords :

These passwords are required for administrative access, database, RabbitMQ, and OpenStack services. Replace “massin” with a secure and unique password in production environments.

=> Set Host ip:

set The <**HOST\_IP**> with your system's local IP (e.g., 192.168.56.x)



```
GNU nano 6.2 /opt/stack/devstack/local.conf
# Note that if 'localrc' is present it will be used in favor of this section.
[[local|localrc]]

# Minimal Contents
# -----

# While 'stack.sh' is happy to run without 'localrc', devlife is better when
# there are a few minimal variables set:

# If the 'ADMIN_PASSWORD' variables are not set here you will be prompted to enter
# values for them by 'stack.sh' and they will be added to 'local.conf'.
ADMIN_PASSWORD=massin
DATABASE_PASSWORD=$ADMIN_PASSWORD
RABBIT_PASSWORD=$ADMIN_PASSWORD
SERVICE_PASSWORD=$ADMIN_PASSWORD

# 'HOST_IP' and 'HOST_IPV6' should be set manually for best results if
# the NIC configuration of the host is unusual, i.e. 'eth1' has the default
# route but 'eth0' is the public interface. They are auto-detected in
# 'stack.sh' but often is indeterminate on later runs due to the IP moving
# from an Ethernet interface to a bridge on the host. Setting it here also
# makes it available for 'openrc' to include when setting 'OS_AUTH_URL'.
# Neither is set by default.
HOST_IP=192.168.56.102
HOST_IPV6=2001:db8::7

# Logging
# -----

# By default 'stack.sh' output only goes to the terminal where it runs. It can
# be configured to additionally log to a file by setting 'LOGFILE' to the full
# path of the destination log file. A timestamp will be appended to the given name.

:~: ^G Help      ^O Write Out  ^H Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^A Set Mark
^X Exit      ^R Read File  ^W Replace    ^V Paste      ^J Justify    ^_ Go To Line  ^E Redo      ^M Copy
```

## change the ownership of /opt/devstack

Then, we change the ownership of the /opt/devstack directory and all of its contents to the user 'stack' and the group 'stack'. This is because when deploying OpenStack with DevStack, the installation directory requires specific permissions for the stack user to ensure the proper functioning of the OpenStack services.

```
$ sudo chown -R stack:stack /opt/stack
```

```
stack@massin-VirtualBox:~/devstack$ sudo chown -R stack:stack /opt/stack
stack@massin-VirtualBox:~/devstack$
```

Then, we run the stack.sh script:

```
stack@massin-VirtualBox:~/devstack$ sudo chown -R stack:stack /opt/stack
stack@massin-VirtualBox:~/devstack$ ./stack.sh
+ unset GREP_OPTIONS
+ unset LANG
+ unset LANGUAGE
+ LC_ALL=en_US.utf8
+ export LC_ALL
++ env
++ grep -E '^OS_'
++ cut -d = -f 1
+ unset
+ umask 022
+ PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin:/usr/local/bin:/usr/sbin:/sbin
+++ dirname ./stack.sh
++ cd .
++ pwd
+ TOP_DIR=/opt/stack/devstack
+ NOUNSET=
+ [[ -n '' ]]
++ date +%s
+ DEVSTACK_START_TIME=1733885670
+ [[ -r /opt/stack/devstack/.stackenv ]]
+ FILES=/opt/stack/devstack/files
+ '[' '!' -d /opt/stack/devstack/files ']'
+ '[' '!' -d /opt/stack/devstack/inc ']'
+ '[' '!' -d /opt/stack/devstack/lib ']'
+ [[ '' == \y ]]
+ [[ 1001 -eq 0 ]]
+ [[ -n '' ]]
+ [[ -e /opt/stack/.no-devstack ]]
```

```
Unpacking liburing2:amd64 (2.1-2build1) ...
Selecting previously unselected package libusbredirparser1:amd64.
Preparing to unpack .../27-libusbredirparser1_0.11.0-2build1_amd64.deb ...
Unpacking libusbredirparser1:amd64 (0.11.0-2build1) ...
Selecting previously unselected package libvirglrenderer1:amd64.
Preparing to unpack .../28-libvirglrenderer1_0.9.1-1-exp1ubuntu2_amd64.deb ...
Unpacking libvirglrenderer1:amd64 (0.9.1-1-exp1ubuntu2) ...
Selecting previously unselected package libfdt1:amd64.
Preparing to unpack .../29-libfdt1_1.6.1-1_amd64.deb ...
Unpacking libfdt1:amd64 (1.6.1-1) ...
Selecting previously unselected package qemu-system-common.
Preparing to unpack .../30-qemu-system-common_1%3a6.2+dfsg-2ubuntu6.24_amd64.deb ...
Unpacking qemu-system-common (1:6.2+dfsg-2ubuntu6.24) ...
Selecting previously unselected package qemu-system-data.
Preparing to unpack .../31-qemu-system-data_1%3a6.2+dfsg-2ubuntu6.24_all.deb ...
Unpacking qemu-system-data (1:6.2+dfsg-2ubuntu6.24) ...
Selecting previously unselected package seabios.
Preparing to unpack .../32-seabios_1.15.0-1_all.deb ...
Unpacking seabios (1.15.0-1) ...
Selecting previously unselected package qemu-system-x86.
Preparing to unpack .../33-qemu-system-x86_1%3a6.2+dfsg-2ubuntu6.24_amd64.deb ...
Unpacking qemu-system-x86 (1:6.2+dfsg-2ubuntu6.24) ...
Selecting previously unselected package qemu-utils.
Preparing to unpack .../34-qemu-utils_1%3a6.2+dfsg-2ubuntu6.24_amd64.deb ...
Unpacking qemu-utils (1:6.2+dfsg-2ubuntu6.24) ...
Selecting previously unselected package qemu-block-extra.
Preparing to unpack .../35-qemu-block-extra_1%3a6.2+dfsg-2ubuntu6.24_amd64.deb ...
Unpacking qemu-block-extra (1:6.2+dfsg-2ubuntu6.24) ...
Selecting previously unselected package qemu-system-gui.
Preparing to unpack .../36-qemu-system-gui_1%3a6.2+dfsg-2ubuntu6.24_amd64.deb ...
Unpacking qemu-system-gui (1:6.2+dfsg-2ubuntu6.24) ...
Selecting previously unselected package ovmf.
Preparing to unpack .../37-ovmf_2022.02-3ubuntu0.22.04.3_all.deb ...
Unpacking ovmf (2022.02-3ubuntu0.22.04.3) ...
Setting up libibverbs1:amd64 (39.0-1) ...
```

| db         | op        | count |
|------------|-----------|-------|
| keystone   | SELECT    | 20462 |
| keystone   | UPDATE    | 7     |
| keystone   | INSERT    | 91    |
| neutron    | DESCRIBE  | 2     |
| neutron    | CREATE    | 1     |
| neutron    | SHOW      | 4     |
| neutron    | SELECT    | 6072  |
| neutron    | INSERT    | 4121  |
| neutron    | DELETE    | 29    |
| neutron    | UPDATE    | 131   |
| placement  | SELECT    | 30    |
| placement  | INSERT    | 66    |
| placement  | SET       | 2     |
| nova_api   | SELECT    | 50    |
| nova_cell1 | SELECT    | 108   |
| nova_cell0 | SELECT    | 33    |
| nova_cell0 | INSERT    | 5     |
| nova_cell0 | UPDATE    | 8     |
| nova_cell1 | INSERT    | 4     |
| nova_cell1 | UPDATE    | 46    |
| cinder     | SELECT    | 57    |
| placement  | UPDATE    | 3     |
| cinder     | INSERT    | 5     |
| cinder     | UPDATE    | 5     |
| nova_api   | INSERT    | 20    |
| glance     | INSERT    | 14    |
| cinder     | DELETE    | 1     |
| glance     | SELECT    | 28    |
| glance     | UPDATE    | 2     |
| nova_api   | SAVEPOINT | 10    |
| nova_api   | RELEASE   | 10    |

|            |           |    |
|------------|-----------|----|
| nova_cell1 | INSERT    | 4  |
| placement  | UPDATE    | 3  |
| cinder     | SELECT    | 59 |
| nova_api   | SAVEPOINT | 10 |
| nova_api   | INSERT    | 20 |
| nova_api   | RELEASE   | 10 |
| cinder     | INSERT    | 5  |
| cinder     | UPDATE    | 7  |
| glance     | INSERT    | 14 |
| cinder     | DELETE    | 1  |
| glance     | SELECT    | 28 |
| glance     | UPDATE    | 2  |
| nova_cell1 | DELETE    | 1  |

This is your host IP address: 192.168.56.102  
This is your host IPv6 address: ::1  
Horizon is now available at <http://192.168.56.102/dashboard>  
Keystone is serving at <http://192.168.56.102/identity/>  
The default users are: admin and demo  
The password: massin

WARNING:  
Configuring uWSGI with a WSGI file is deprecated, use module paths instead

Services are running under systemd unit files.  
For more information see:  
<https://docs.openstack.org/devstack/latest/systemd.html>

DevStack Version: 2025.1  
Change: 05f7d302cfa2da73b2887afcde92ef65b1001194 lib/cinder: Migrate cinder to WSGI module path 2024-12-09 14:03:49 +0000  
OS Version: Ubuntu 22.04 jammy

stack@massin-VirtualBox:~/devstack\$

The **stack.sh** script is the main script used by DevStack to deploy an OpenStack environment on a machine. When you run the stack.sh script, several steps are performed, including:

**System Configuration:** The script begins by checking and configuring various system settings to ensure that the machine is ready for OpenStack deployment. This may include network configuration, checking software dependencies, etc.

**Downloading OpenStack Components:** The script downloads the required OpenStack components from the appropriate Git repositories. This includes services such as Nova (compute), Neutron (networking), Glance (image service), Keystone (identity), and others.

**Configuring OpenStack Services:** Once the components are downloaded, the stack.sh script configures these services according to the parameters specified in the local.conf configuration file. This may include parameters such as IP addresses, passwords, and choices of services to enable, etc.

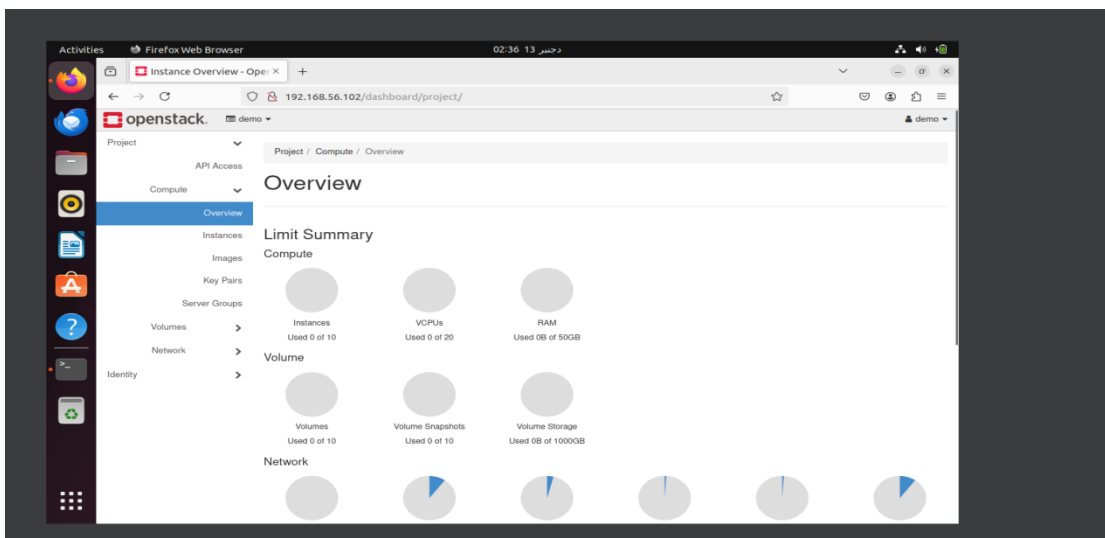
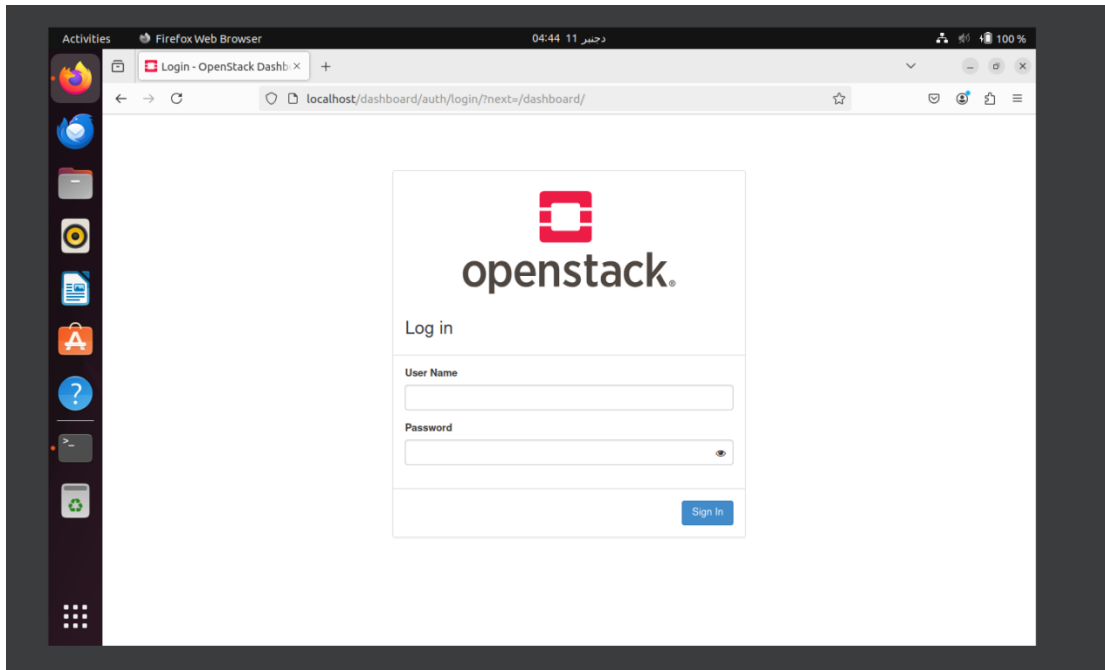
**Starting OpenStack Services:** Once configured, the OpenStack services are started. This means that the various services, such as Nova, Neutron, and others, are launched as processes on the machine.

**Creating Projects, Users, and Roles:** The script typically creates projects, users, and roles in OpenStack to simulate a multi-user environment.

**Creating Images:** Images of instances (virtual machines) may be downloaded and saved for use by the deployed instances.

**Configuring the Client Environment:** The script often generates an openrc or stackrc file that can be sourced in the current terminal environment to configure the necessary environment variables to interact with the newly deployed OpenStack environment.

DevStack typically installs the Horizon dashboard, which is a web interface for OpenStack. You can access the dashboard by opening a web browser and going to the IP address of your machine (by default, this is often <http://localhost/dashboard>).



Next, we need to download the script **admin-openrc.sh** from the OpenStack graphical interface so that we can display all the installed services using the following commands:

#### **4. Configures the environment of OpenStack**

Then, the **admin-openrc.sh** script must be downloaded from the OpenStack graphical interface so that we can display all the installed services.

The file **admin-openrc.sh** is OpenStack RC file (resource configuration file) used to

set environment variables required for interacting with OpenStack services via the command line. file typically corresponds to a specific user and project with different roles and permissions.

## **5. Key Differences Between admin-openrc.sh and demo-openrc.sh**

### **User Role and Permissions:**

- `admin-openrc.sh`:
  - Used for administrative tasks.
  - Configures the environment variables for a user with administrative privileges (usually the `admin` user).
  - Allows full access to OpenStack services, including creating projects, users, roles, and managing other resources.
- `demo-openrc.sh`:
  - Used for a regular (non-administrative) user, typically a demo or test user.
  - Configures environment variables for a user with restricted access based on the assigned roles.
  - Limited to managing resources (like instances, volumes, and networks) within a specific project.

### **Scope of Operations:**

- The `admin-openrc.sh` file gives access to all projects and administrative APIs.
- The `demo-openrc.sh` file is scoped to a specific project (e.g., `demo`), and the user is restricted to their project's resources.

### **Environment Variables:**

- Both files set variables like `OS_USERNAME`, `OS_PROJECT_NAME`, `OS_PASSWORD`, `OS_AUTH_URL`, and others.
- The values differ depending on the user and project associated with the file

Example snippet from `admin-openrc.sh`:

```
export OS_USERNAME=admin
export OS_PROJECT_NAME=admin
export OS_PASSWORD=your_admin_password
export OS_AUTH_URL=http://controller:5000/v3
```

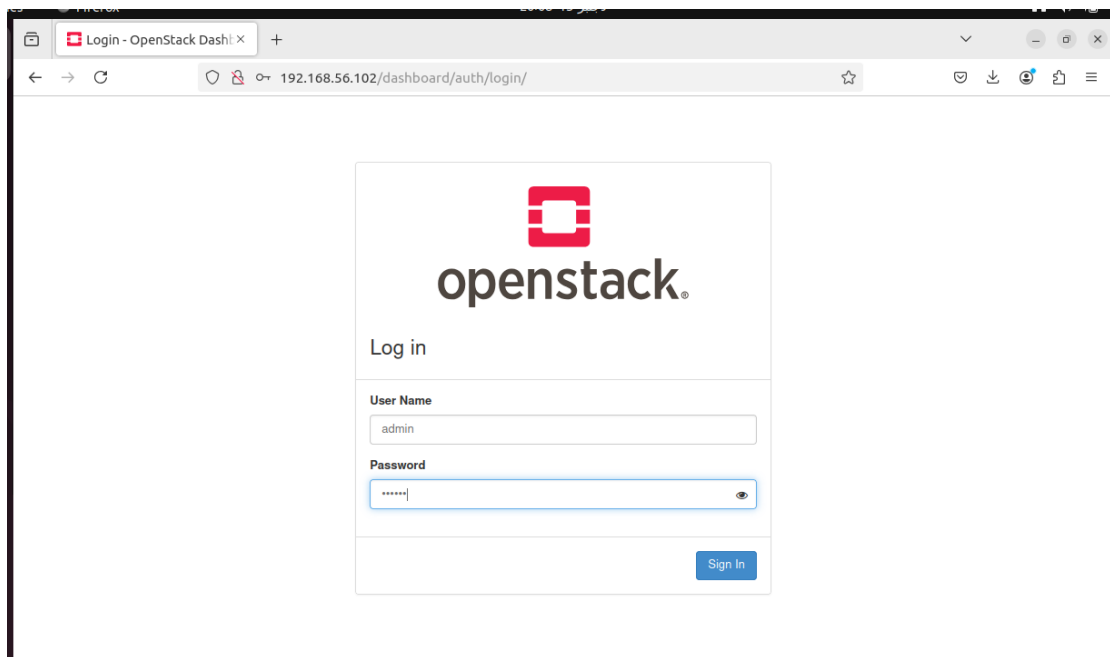
Example snippet from `demo-openrc.sh`:

```
export OS_USERNAME=demo
export OS_PROJECT_NAME=demo
export OS_PASSWORD=your_demo_password
export OS_AUTH_URL=http://controller:5000/v3
```

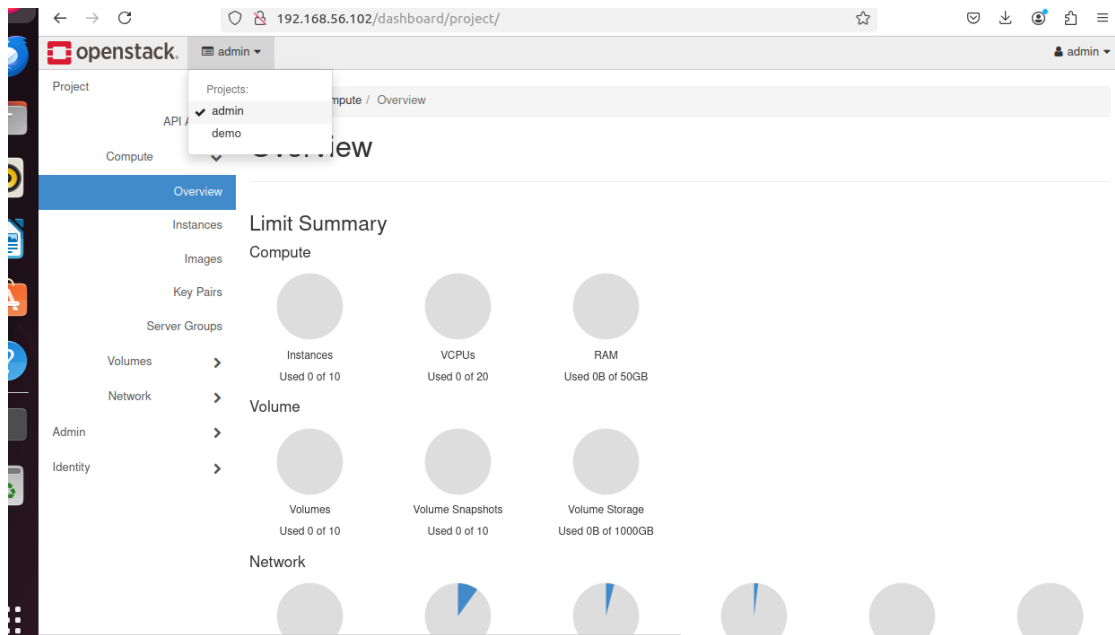
- Use admin-openrc.sh. This is required for tasks like creating new projects, managing users, assigning roles, or setting quotas.
- Use demo-openrc.sh if you're only working within the demo project and don't need administrative-level operations.

## Steps :

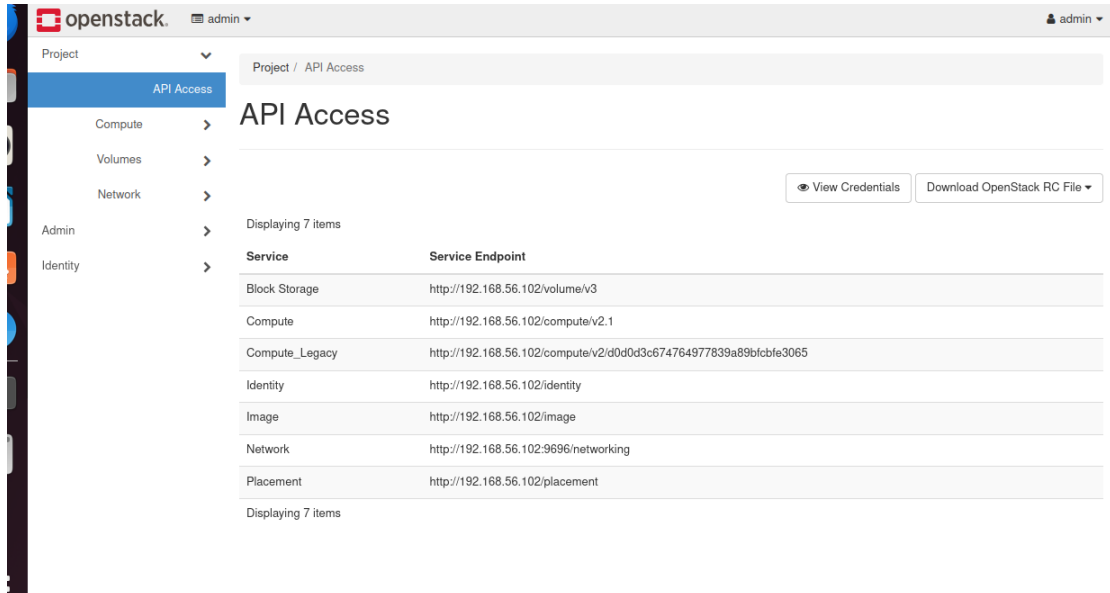
1. Connect to dashboard using admin credentials:



## 2. Chose admin project:

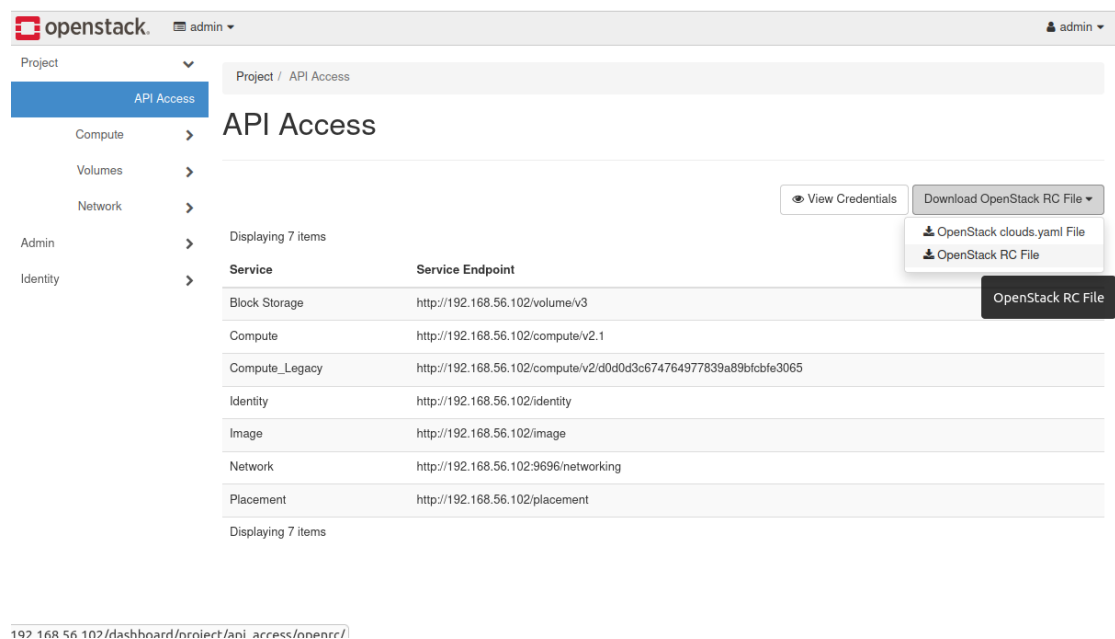


## 3. Access “Api Access” Interface :

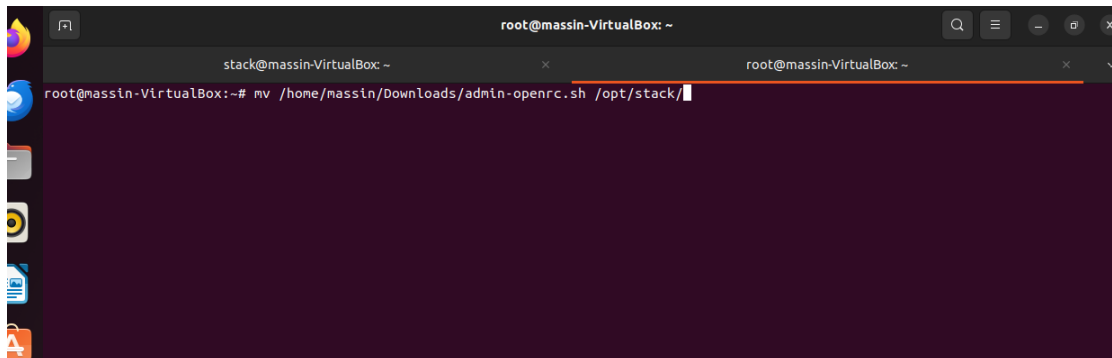




#### 4. Install admin-openrc.sh file:



#### 5. Move the file into stack home directory:



## 6. Source the file :

```
stack@massin-VirtualBox: ~  
stack@massin-VirtualBox:~$ ls  
admin-openrc.sh  bin          cinder  devstack  glance  keystone  neutron  novnc  placement  tempest  
async            bindenv      data    devstack.subunit  horizon  logs      nova     os-test-images  requirements  
stack@massin-VirtualBox:~$ source admin-openrc.sh  
Please enter your OpenStack Password for project admin as user admin:  
stack@massin-VirtualBox:~$
```

Sourcing the RC file (e.g., `admin-openrc.sh` or `demo-openrc.sh`) sets environment variables required for authentication and interaction with OpenStack services via the command-line interface. These variables include details like the username, project name, password, and authentication endpoint, which eliminate the need to specify them manually for each command. By sourcing the RC file, the user gains access to OpenStack resources and permissions based on their role and project scope. For example, the `admin-openrc.sh` provides administrative privileges, while the `demo-openrc.sh` limits access to resources within the `demo` project.

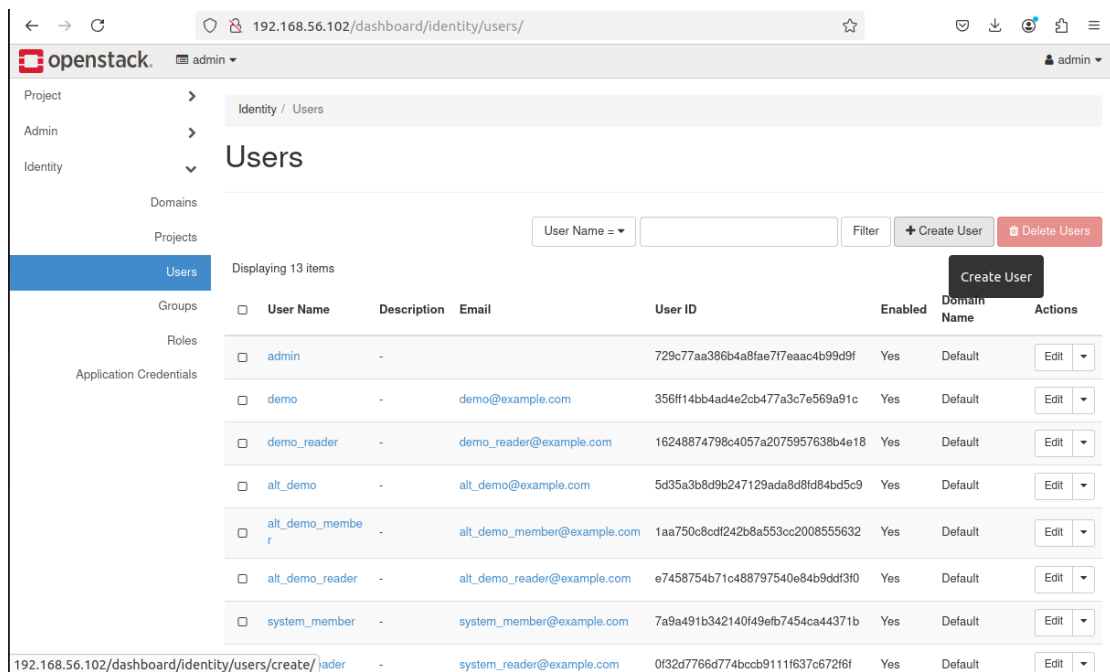
Run Command [ `openstack service list` ]

```
Please enter your OpenStack Password for project admin as user admin:  
stack@massin-VirtualBox:~$ openstack service list  
+-----+-----+-----+  
| ID | Name | Type |  
+-----+-----+-----+  
| 0c7fd3bf0fb44280a936f1c27c95f7e0 | keystone | identity |  
| 129c685bacb8458aabdf22a83b9591ba | placement | placement |  
| 2863a3f0427b4c12b99af7235a4f71ef | glance | image |  
| 2d536beb55a84998a9d4c2df7f9e31a4 | cinder | block-storage |  
| 565020a076aa42748c29ee0de93ce764 | nova | compute |  
| 6ed35391cbda40838a20bdaad4172e46 | neutron | network |  
| bab610e340194ca297bbda0f5bdf01c6 | nova_legacy | compute_legacy |  
+-----+-----+-----+  
stack@massin-VirtualBox:~$
```

is used to display a list of all services registered in the OpenStack Identity service (Keystone). It provides information about the services that are available in the OpenStack environment and their corresponding API endpoints.

# VI. Creation of instances

## 1. Creation of an Ubuntu instance:



The screenshot shows the OpenStack Identity Users interface. The left sidebar contains a navigation menu with 'Project', 'Admin', and 'Identity' sections. The 'Identity' section is expanded, showing 'Domains', 'Projects', 'Users', 'Groups', 'Roles', and 'Application Credentials'. The 'Users' section is selected. The main content area displays a table of users with columns: User Name, Description, Email, User ID, Enabled, Domain Name, and Actions. There are 13 items displayed. A 'Create User' button is visible in the top right corner of the table area.

| User Name       | Description | Email                       | User ID                          | Enabled | Domain Name | Actions |
|-----------------|-------------|-----------------------------|----------------------------------|---------|-------------|---------|
| admin           | -           |                             | 729c77aa386b4a8fae717eaac4b99d9f | Yes     | Default     | Edit    |
| demo            | -           | demo@example.com            | 356ff14bb4ad4e2cb477a3c7e569a91c | Yes     | Default     | Edit    |
| demo_reader     | -           | demo_reader@example.com     | 16248874798c4057a2075957638b4e18 | Yes     | Default     | Edit    |
| alt_demo        | -           | alt_demo@example.com        | 5d35a3b8d9b247129ada8d8fd84bd5c9 | Yes     | Default     | Edit    |
| alt_demo_member | -           | alt_demo_member@example.com | 1aa750c8cdf242b8a553cc2008555632 | Yes     | Default     | Edit    |
| alt_demo_reader | -           | alt_demo_reader@example.com | e7458754b71c488797540e84b9dd3f0  | Yes     | Default     | Edit    |
| system_member   | -           | system_member@example.com   | 7a9a491b342140f49efb7454ca44371b | Yes     | Default     | Edit    |
| system_reader   | -           | system_reader@example.com   | 0f32d7766d774bccb9111f637c672f6f | Yes     | Default     | Edit    |

Connect as Admin user and then navigate to “Users” interface under “identity” section , and creat a new user .

openstack. admin 192.168.56.102/dashboard/identity/users/

Default

**User Name \***

Tdia\_user

**Description**

A hard worker engineer, who believes that he can become rich one day as an employee

**Email**

**Password \***

test

**Confirm Password \***

test

**Primary Project**

Select a project

**Role**

member

☒ **Enabled**

☐ **Lock password**

+ Create User Delete

| Enabled | Domain Name | Action |
|---------|-------------|--------|
| Default | Edit        |        |
| Default | Edit        |        |
| Default | Edit        |        |
| Default | Edit        |        |
| Default | Edit        |        |
| Default | Edit        |        |
| Default | Edit        |        |
| Default | Edit        |        |
| Default | Edit        |        |

openstack. admin 192.168.56.102/dashboard/identity/users/

admin

Project

Admin

Identity

Domains

Projects

Users

Groups

Roles

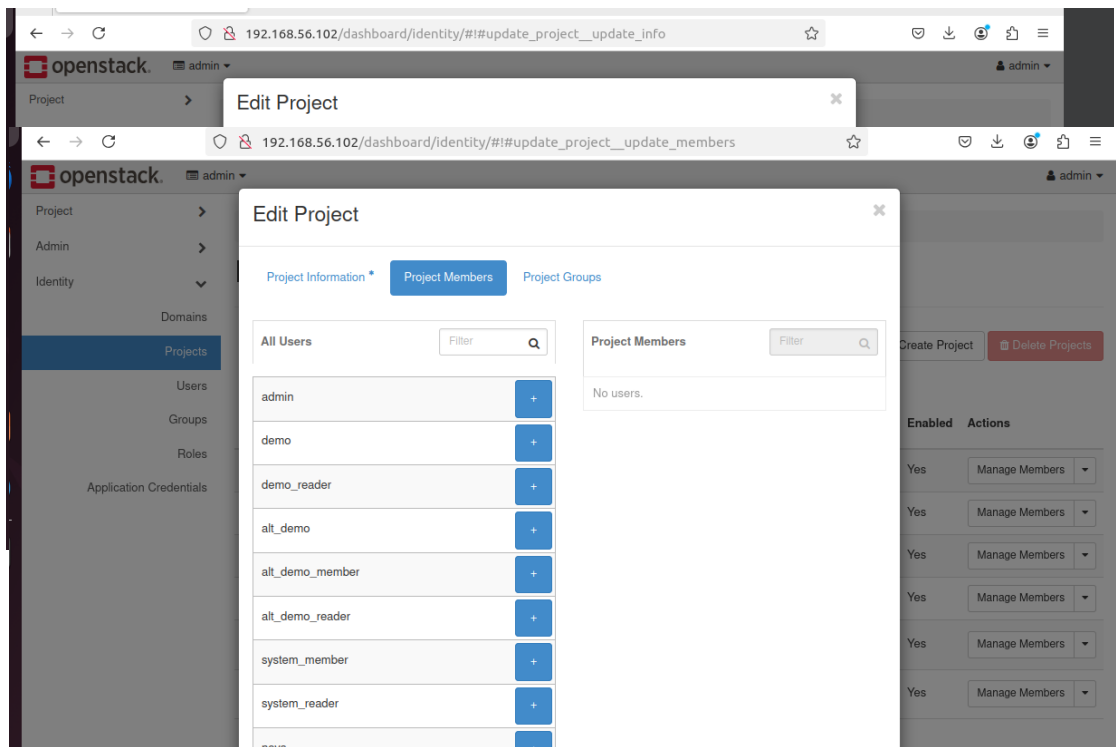
Application Credentials

|                          |                 |                                                                                     |                             |                                  |     |         |      |  |
|--------------------------|-----------------|-------------------------------------------------------------------------------------|-----------------------------|----------------------------------|-----|---------|------|--|
| <input type="checkbox"/> | alt_demo_member | -                                                                                   | alt_demo_member@example.com | 1aa750c8cdf242b8a553cc2008555632 | Yes | Default | Edit |  |
| <input type="checkbox"/> | alt_demo_reader | -                                                                                   | alt_demo_reader@example.com | e7458754b71c488797540e84b9ddf310 | Yes | Default | Edit |  |
| <input type="checkbox"/> | system_member   | -                                                                                   | system_member@example.com   | 7a9a491b342140f49efb7454ca44371b | Yes | Default | Edit |  |
| <input type="checkbox"/> | system_reader   | -                                                                                   | system_reader@example.com   | 0132d7766d774bccb9111f637c67216f | Yes | Default | Edit |  |
| <input type="checkbox"/> | nova            | -                                                                                   |                             | 6e7cebe2da8c41fe9750e847b70fe1e5 | Yes | Default | Edit |  |
| <input type="checkbox"/> | glance          | -                                                                                   |                             | 71b1f3eb91b44eec8bdc8edfdb212a6e | Yes | Default | Edit |  |
| <input type="checkbox"/> | cinder          | -                                                                                   |                             | 4d5f486a751143d3be5375f6e669221e | Yes | Default | Edit |  |
| <input type="checkbox"/> | neutron         | -                                                                                   |                             | a714104b2ec54ddd3d9207039dbde42  | Yes | Default | Edit |  |
| <input type="checkbox"/> | placement       | -                                                                                   |                             | c45fa70455ec4f80ba5b8e2af425b996 | Yes | Default | Edit |  |
| <input type="checkbox"/> | Tdia_user       | A hard worker engineer, who believes that he can become rich one day as an employee |                             | c01a3a25fa06479db7c5dca8d6a616c1 | Yes | Default | Edit |  |

Displaying 14 items

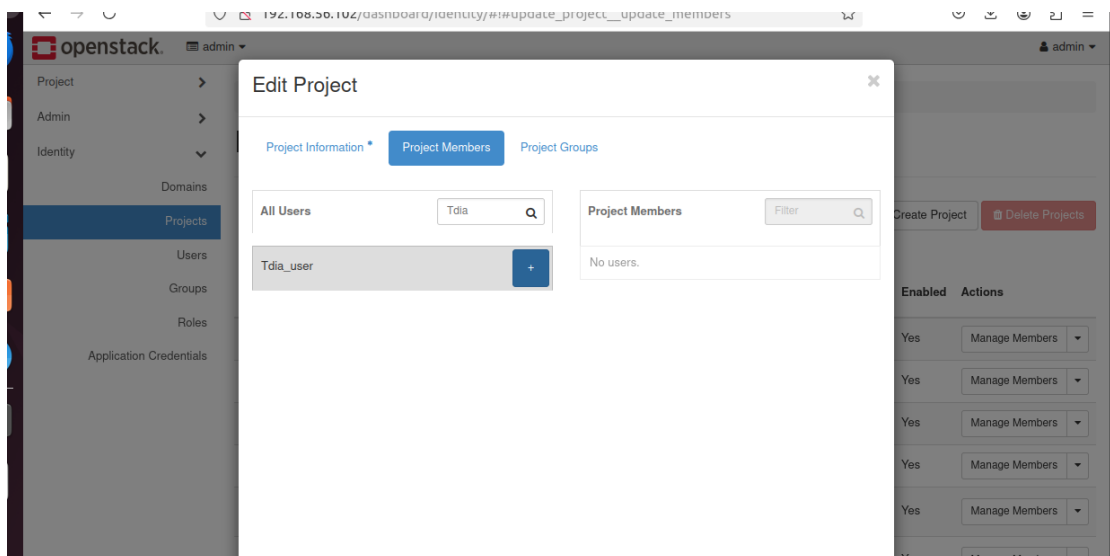
## 2. Creation of a New Project:

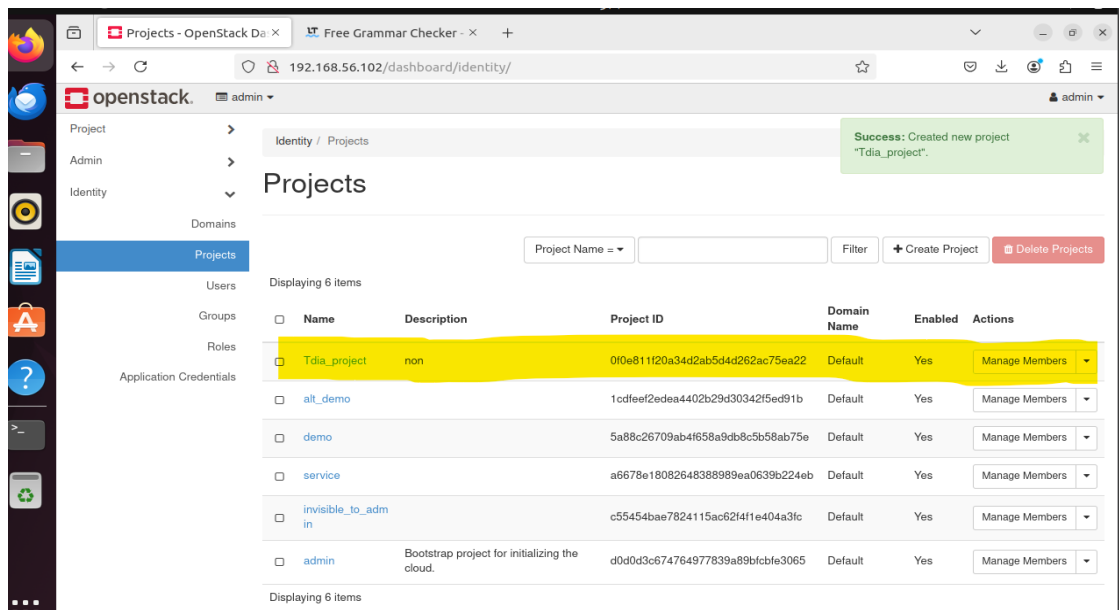
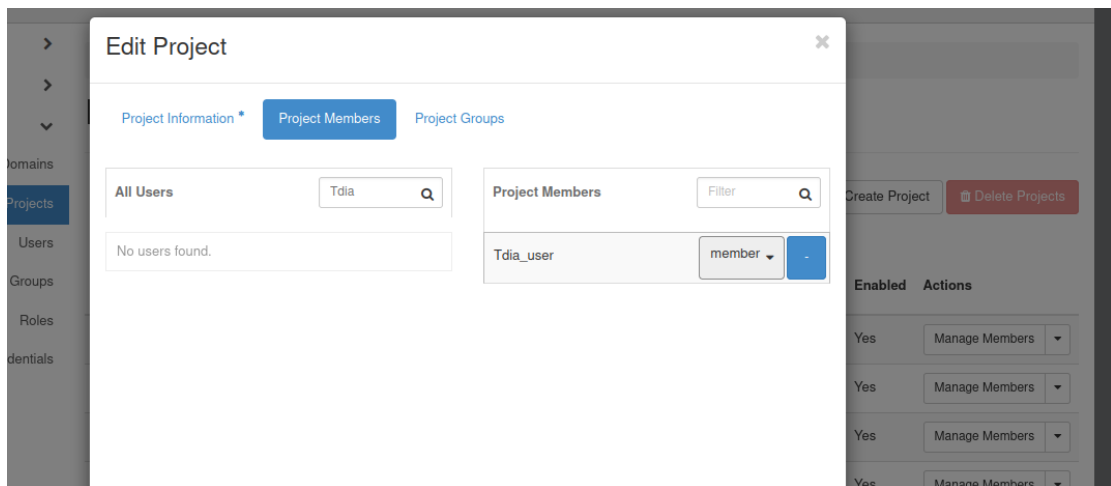
Set a Name and a description(optional) to the project : Tdia\_user



Set Restriction of Access to the Project :(Only our user will have access to the project)

Affect the User created before to the project :



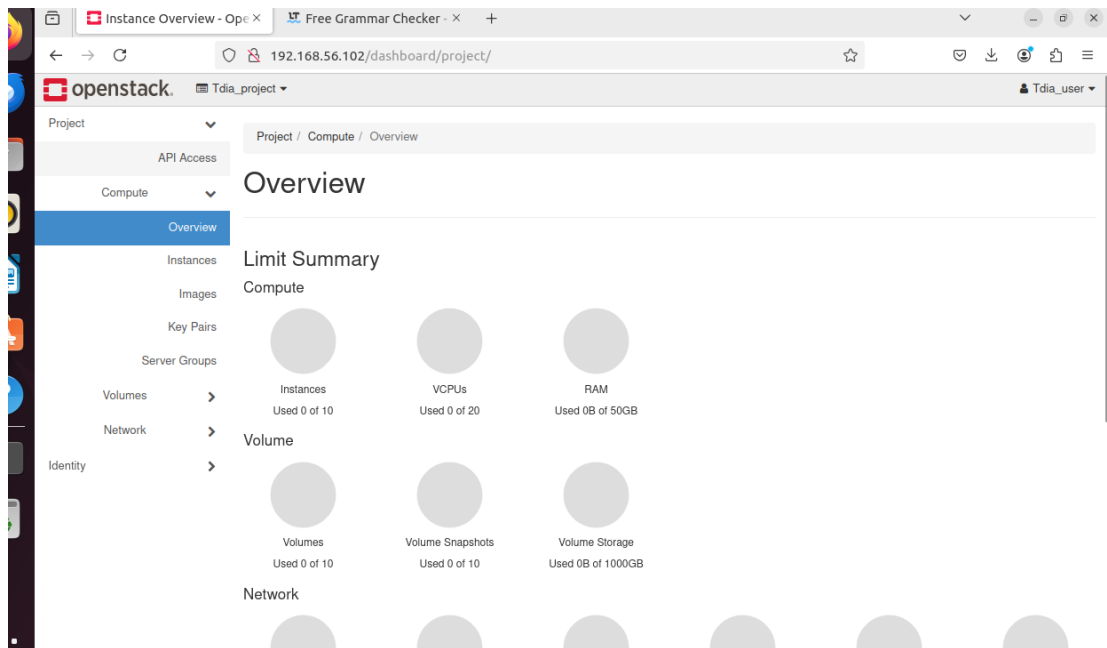


Then, we log in to the New- user account (Tdia\_user) for network configuration before creating our instance:

Log in with user credentials.

The screenshot shows the OpenStack Identity dashboard. On the left, there is a sidebar with navigation links for Project, Admin, and Identity. The main content area is titled 'Projects' and features the OpenStack logo and a 'Log in' section. The login form has two input fields: 'User Name' with the value 'Tdia\_user' and 'Password' with the value 'test'. A 'Sign In' button is located at the bottom right of the login form. To the right of the login form, there is a table listing domains. The table has columns for 'Domain Name', 'Enabled', and 'Actions'. The 'Actions' column contains a 'Manage Members' button for each domain.

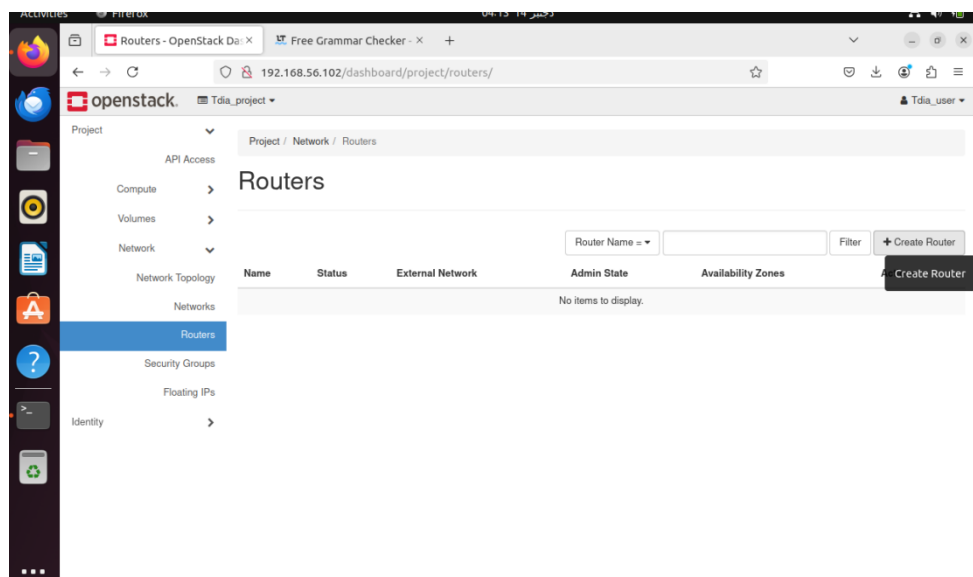
| Domain Name           | Enabled | Actions |                |
|-----------------------|---------|---------|----------------|
| 4d2ab5d4d262ac75ea22  | Default | Yes     | Manage Members |
| 4402b29d30342f5ed91b  | Default | Yes     | Manage Members |
| b4f658a9db8c5b58ab75e | Default | Yes     | Manage Members |
| 648388989ea0639b224eb | Default | Yes     | Manage Members |
| 24115ac62f4f1e404a3fc | Default | Yes     | Manage Members |



Next, we log in to the user account to configure the network before creating our instance.

### 3. Configure Network

#### Step 1 : Creation of a router to interconnect our instance and public network beforehand

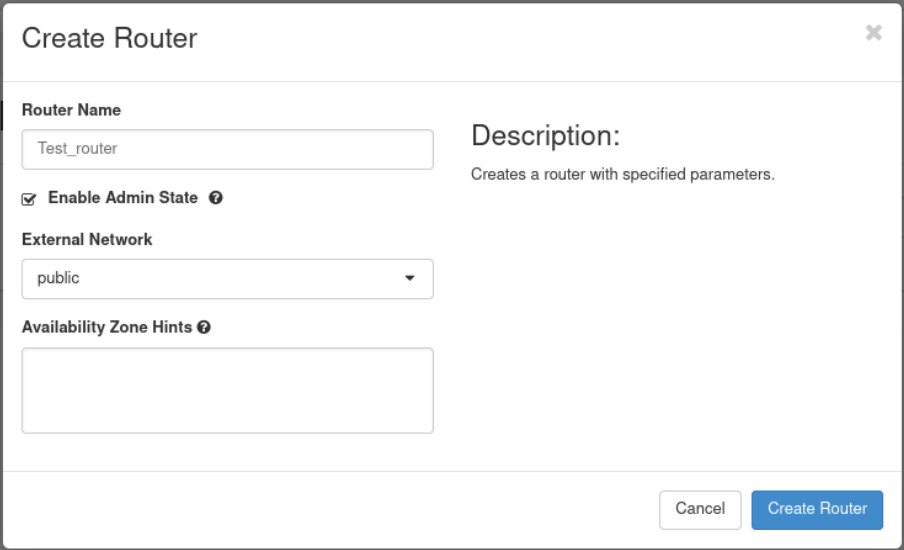


Creation of a new router:

Name: test-router

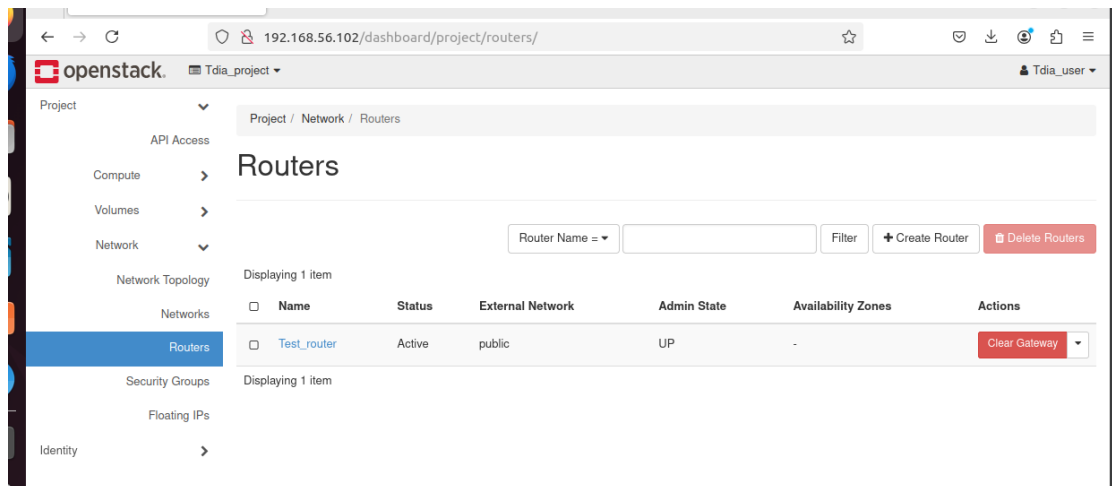


- We will link it to the 'public' network.



The 'Create Router' dialog box is shown. It contains the following fields and options:

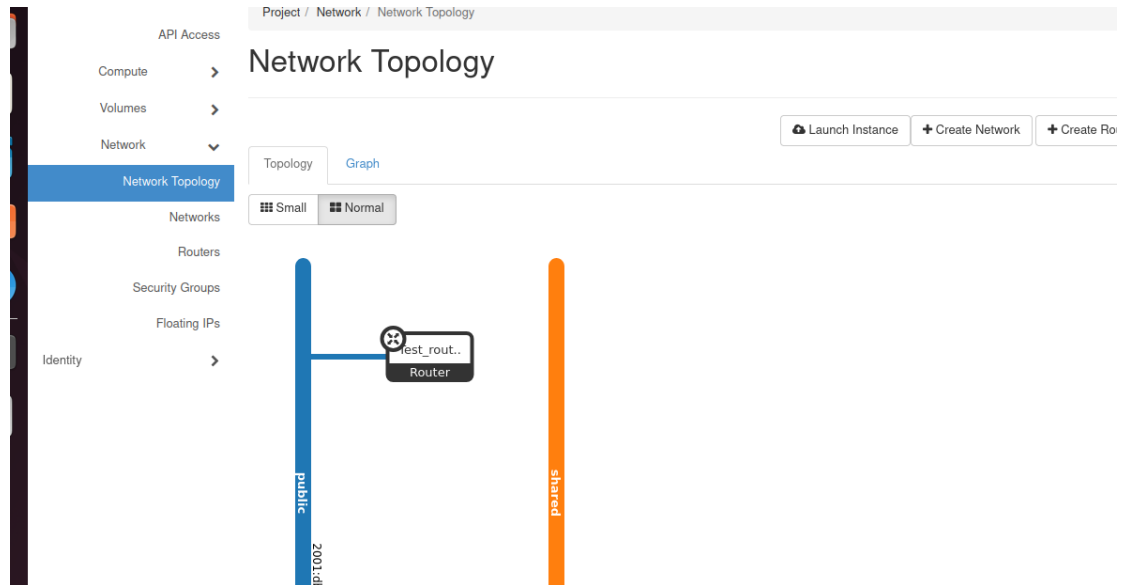
- Router Name:** A text input field containing 'Test\_router'.
- Enable Admin State:** A checkbox that is checked.
- External Network:** A dropdown menu showing 'public'.
- Availability Zone Hints:** An empty text area.
- Description:** A text area containing the text 'Creates a router with specified parameters.'
- Buttons:** 'Cancel' and 'Create Router' buttons at the bottom right.



The OpenStack dashboard shows the 'Routers' page. The left sidebar has a menu with 'Project', 'API Access', 'Compute', 'Volumes', 'Network', 'Network Topology', 'Networks', 'Routers', 'Security Groups', 'Floating IPs', and 'Identity'. The 'Routers' section is selected. The main content area shows a table of routers.

| Name        | Status | External Network | Admin State | Availability Zones | Actions       |
|-------------|--------|------------------|-------------|--------------------|---------------|
| Test_router | Active | public           | UP          | -                  | Clear Gateway |

Our network creating a router :



## Step 2 : Creation of a private subnet

We assign it the name: test-private

The screenshot shows the 'Networks' page in the OpenStack dashboard. The left sidebar is the same as in the previous image. The main area shows a table of networks. Above the table, there are filters and buttons for '+ Create Network' and 'Delete Networks'. A 'Create Network' button is also visible as a tooltip. The table has columns: Name, Subnets Associated, Shared, External, Status, Admin State, Availability Zones, and Actions. There are two rows of data.

| <input type="checkbox"/> | Name   | Subnets Associated                                              | Shared | External | Status | Admin State | Availability Zones | Actions |
|--------------------------|--------|-----------------------------------------------------------------|--------|----------|--------|-------------|--------------------|---------|
| <input type="checkbox"/> | shared | shared-subnet 192.168.233.0/24                                  | Yes    | No       | Active | True        | -                  |         |
| <input type="checkbox"/> | public | ipv6-public-subnet 2001:db8::/64<br>public-subnet 172.24.4.0/24 | No     | Yes      | Active | True        | -                  |         |

-Subnet name: test-private-subnet

We specify the network address following the proposed syntax: **10.10.93.0/24** in IPv4, and for the gateway, we will leave it to be automatically selected.

The screenshot shows the 'Create Network' wizard in the OpenStack dashboard. The 'Network' tab is selected. The 'Network Name' field contains 'test-private'. The 'Enable Admin State' checkbox is checked. The 'Create Subnet' checkbox is also checked. The 'Availability Zone Hints' field is empty. The 'MTU' field is set to the default value. The 'Next' button is highlighted in blue.

The screenshot shows the 'Create Network' wizard in the OpenStack dashboard, now on the 'Subnet' tab. The 'Subnet Name' field contains 'test-private-subnet'. The 'Network Address Source' dropdown is set to 'Enter Network Address manually'. The 'Network Address' field contains '10.10.93.0/24'. The 'IP Version' dropdown is set to 'IPv4'. The 'Gateway IP' field is empty. The 'Disable Gateway' checkbox is unchecked. The 'Next' button is highlighted in blue.

Then, we move on to the DHCP configuration:

For the address range, we provided: 10.10.93.100, 10.10.93.120.

For the DNS (Domain Name System), we selected Google's public DNS: 8.8.8.8.

Create Network

Network Subnet Subnet Details

☒ Enable DHCP

Specify additional attributes for the subnet.

Allocation Pools ⓘ

10.10.93.100,10.10.93.120

DNS Name Servers ⓘ

8.8.8.8

Host Routes ⓘ

Cancel Back Create

Project / Network / Networks

## Networks

Name =  Filter [+ Create Network](#) [Delete Networks](#)

Displaying 3 items

| <input type="checkbox"/> | Name         | Subnets Associated                                             | Shared | External | Status | Admin State | Availability Zones | Actions                      |
|--------------------------|--------------|----------------------------------------------------------------|--------|----------|--------|-------------|--------------------|------------------------------|
| <input type="checkbox"/> | shared       | shared-subnet 192.168.233.0/24                                 | Yes    | No       | Active | True        | -                  |                              |
| <input type="checkbox"/> | test-private | test-private-subnet 10.10.93.0/24                              | No     | No       | Active | True        | -                  | <a href="#">Edit Network</a> |
| <input type="checkbox"/> | public       | ipv6-public-subnet 2001:db8::64<br>public-subnet 172.24.4.0/24 | No     | Yes      | Active | True        | -                  |                              |

Displaying 3 items

Overview of the network :

API Access

Compute

Volumes

Network

Network Topology

Networks

Routers

Security Groups

Floating IPs

Project / Network / Networks / test-private

test-private

OverviewSubnetsPorts

Name

ID

Project ID

Status

Admin State

Shared

External Network

MTU

test-private

0988efd6-02d4-4d86-b5aa-3e1128194d9c

0f0e811f20a34d2ab5d4d262ac75ea22

Active

True

No

No

1442

We can notice that even though we did not specify a Gateway IP, a Gateway IP is provided by the DHCP..

test-private

OverviewSubnetsPorts

Subnets

Filter

+ Create Subnet

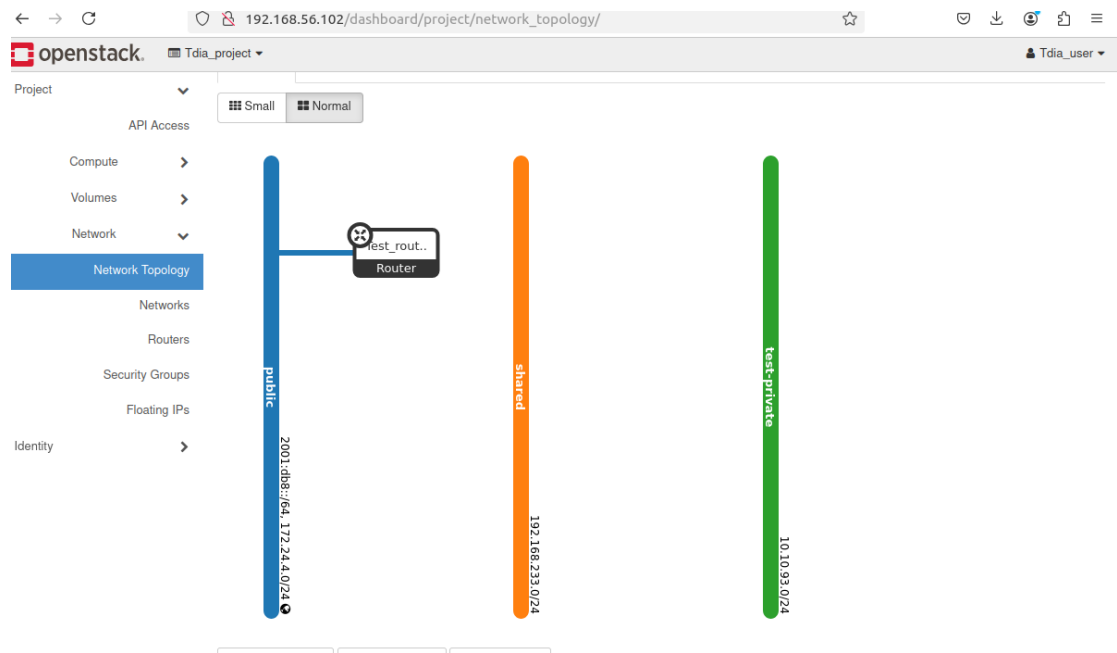
Delete Subnets

Displaying 1 item

| <input type="checkbox"/> | Name                | Network Address | IP Version | Gateway IP | Actions     |
|--------------------------|---------------------|-----------------|------------|------------|-------------|
| <input type="checkbox"/> | test-private-subnet | 10.10.93.0/24   | IPv4       | 10.10.93.1 | Edit Subnet |

Displaying 1 item

Our network after creating the subnet:

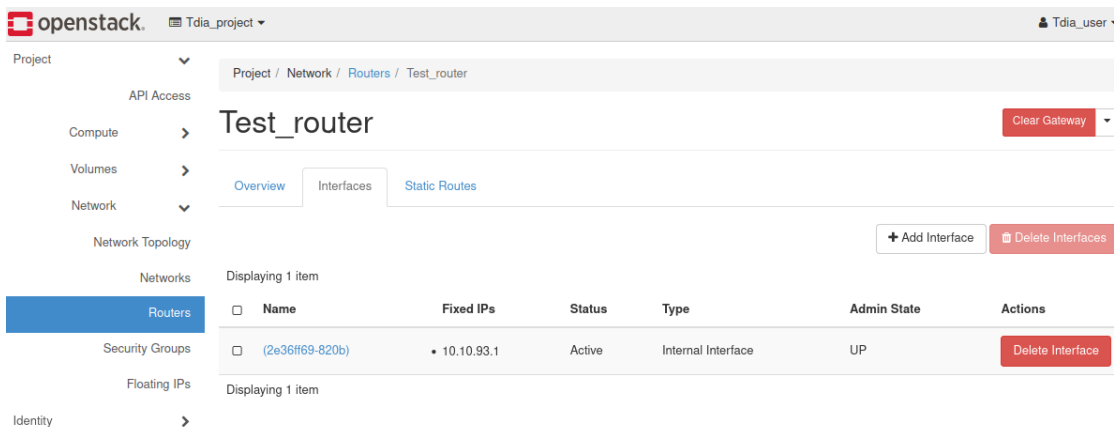
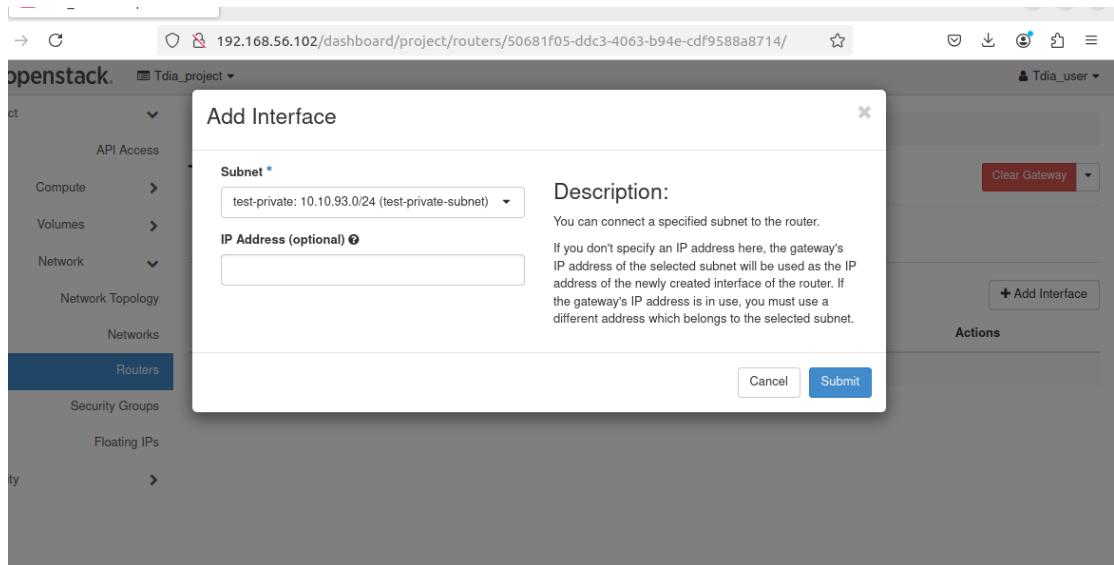


### Step 3: Linking the private network to the router

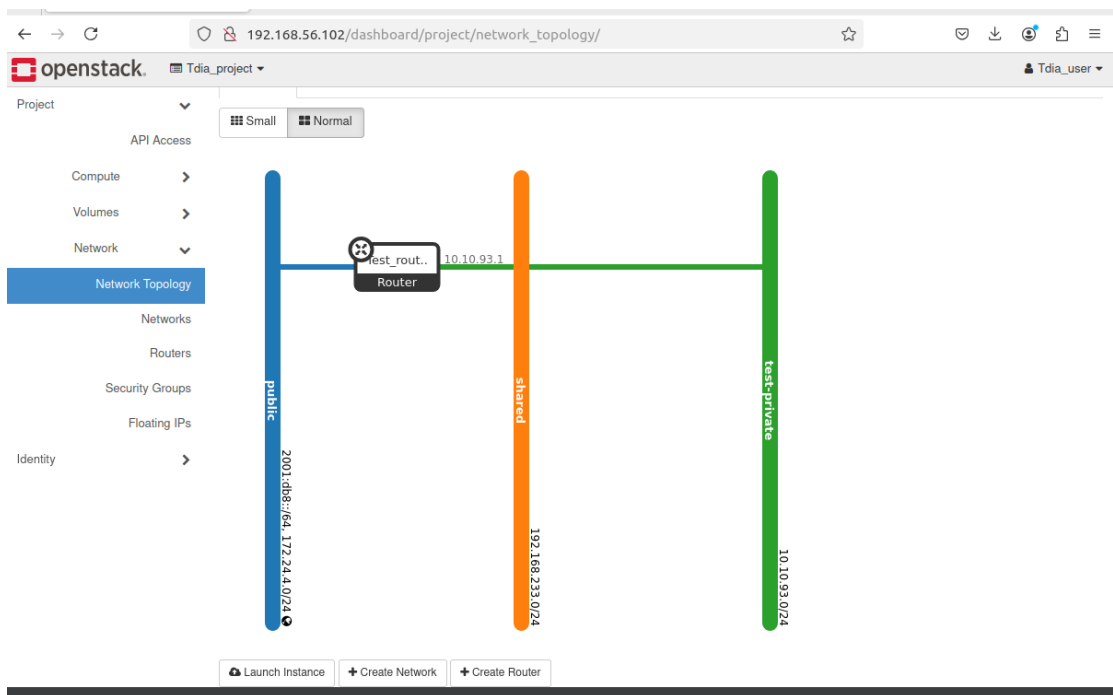
First, we add an interface to our router.

The screenshot shows the 'Test\_router' configuration page in the OpenStack dashboard. The breadcrumb trail is 'Project / Network / Routers / Test\_router'. The page has tabs for 'Overview', 'Interfaces', and 'Static Routes'. The 'Overview' tab is active. Below the tabs, there is a table with columns: Name, Fixed IPs, Status, Type, Admin State, and Actions. The table is currently empty, displaying 'No items to display.' There is a '+ Add Interface' button in the top right corner of the table area. The left sidebar shows the navigation menu with 'Routers' selected. The top of the dashboard shows the OpenStack logo, project name 'Tdla\_project', and user 'Tdla\_user'.

First, we select the subnet we just created:



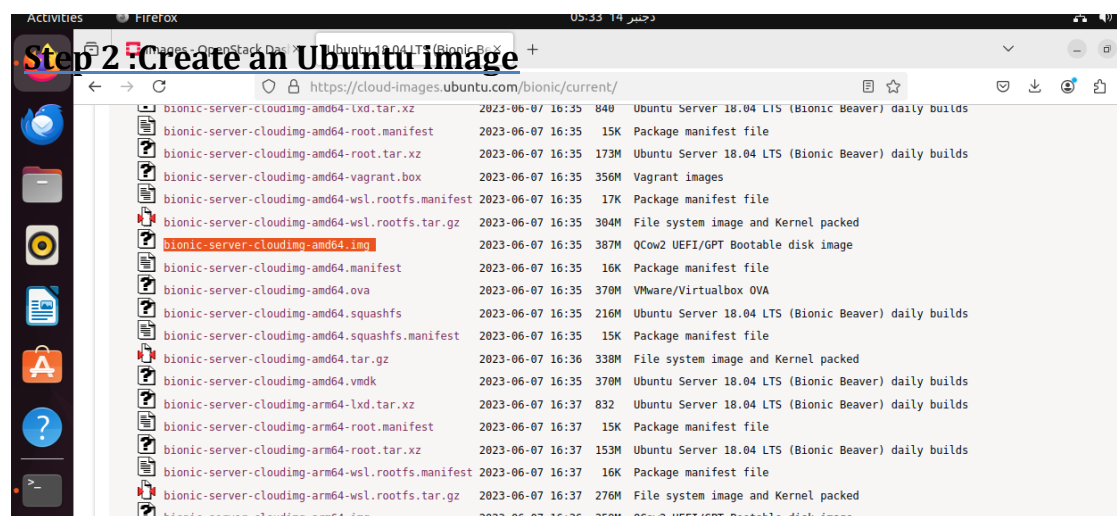
Our network after the connection:  
We can see that our private network is now linked to our public network via the router



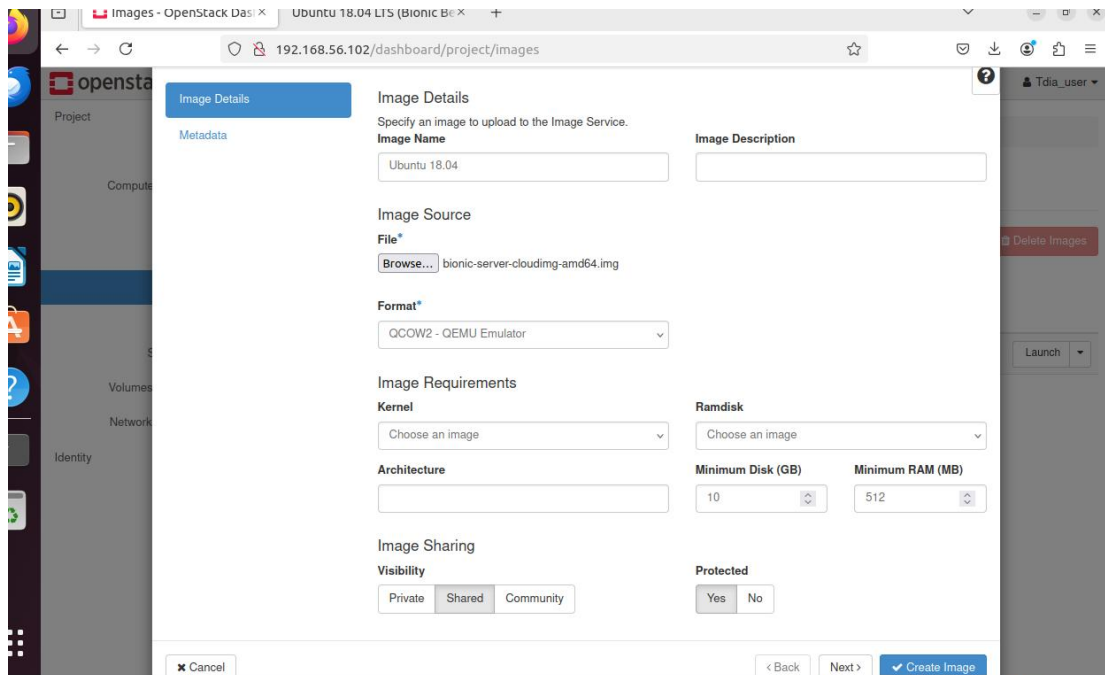
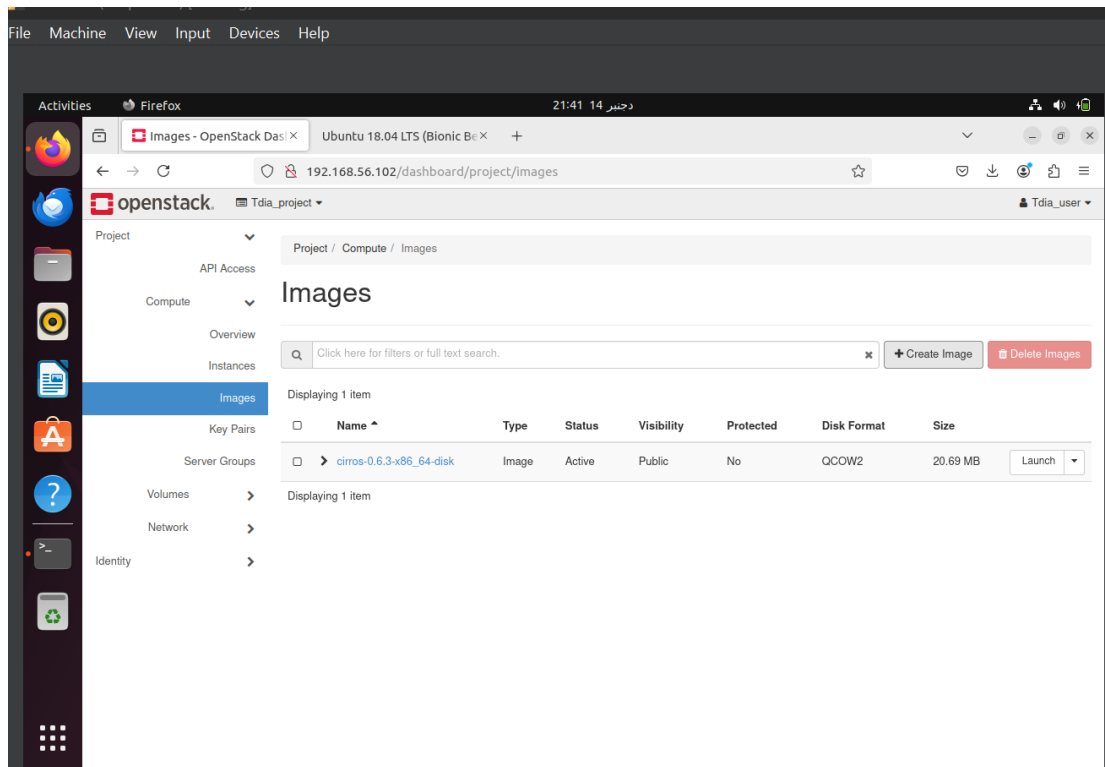
## 1. Create an Instance

### Step 1 : import Ubuntu image 18.04 made by Openstack:

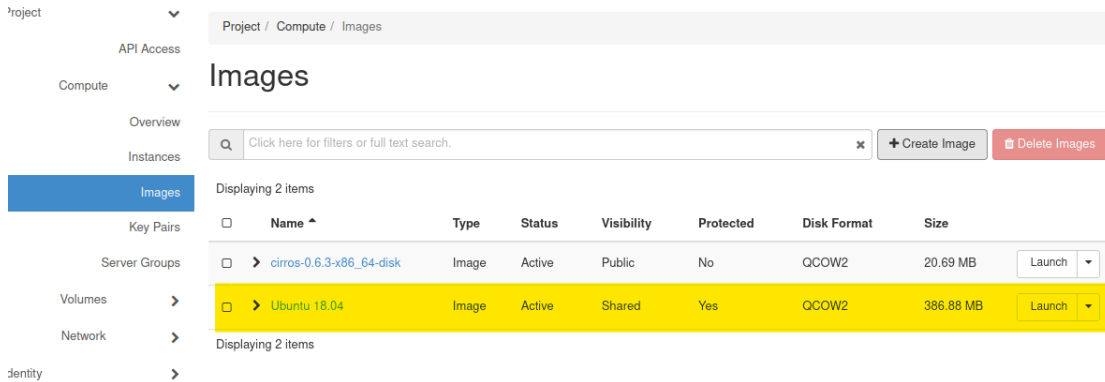
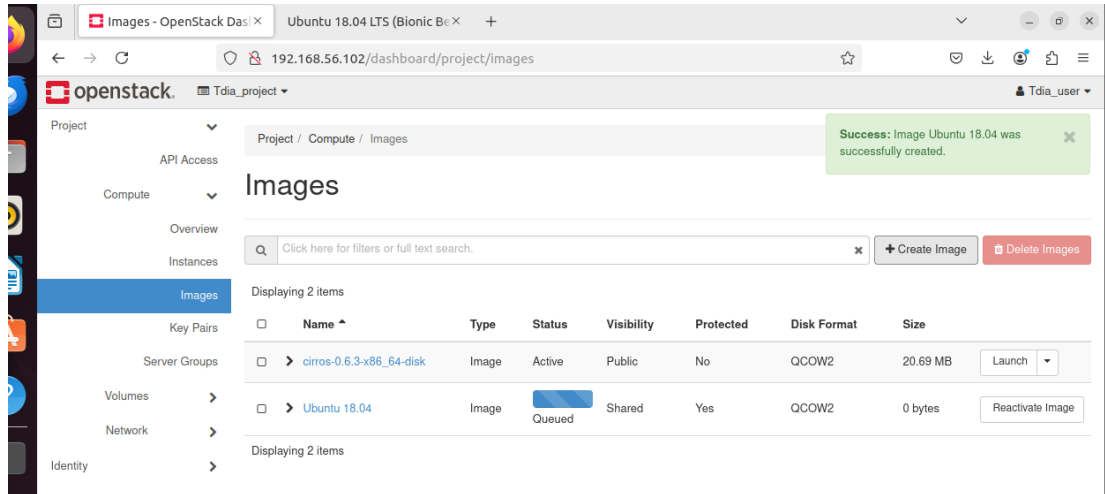
[ <https://cloud-images.ubuntu.com/bionic/current/> ]







**The Glance API** is responsible for managing images in OpenStack.  
The name of the image is set to: ubuntu 18.04.  
The format of the image is specified as qcow2.  
The minimum disk size is set to 10 GB.  
The minimum RAM requirement is 512 MB.  
The image is configured as shared between users and protected to prevent deletion.

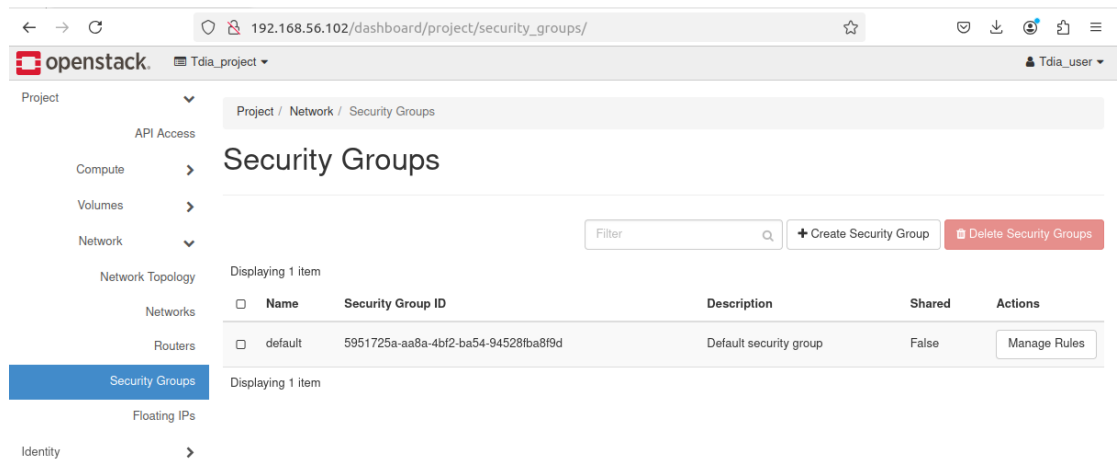


As you can see then image is created successfully.

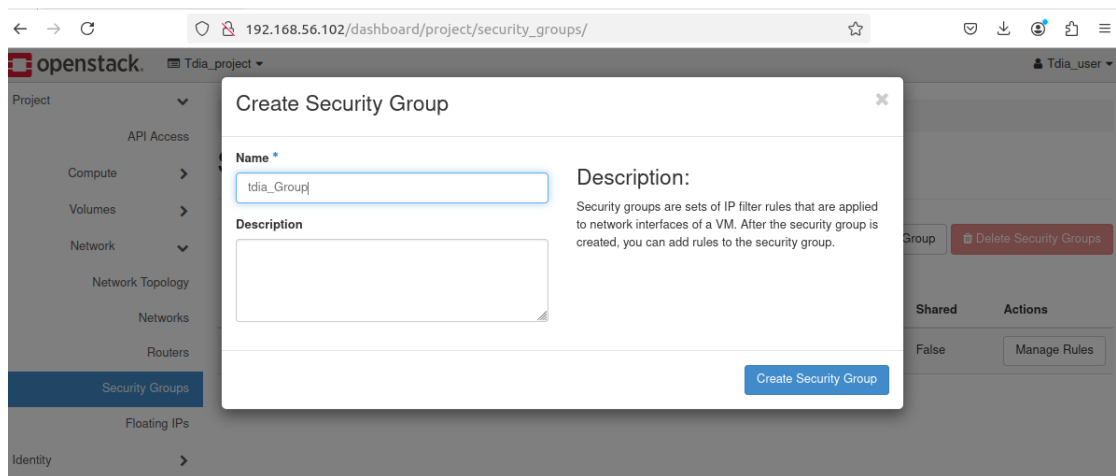
### Step 3: Creating a Security Group

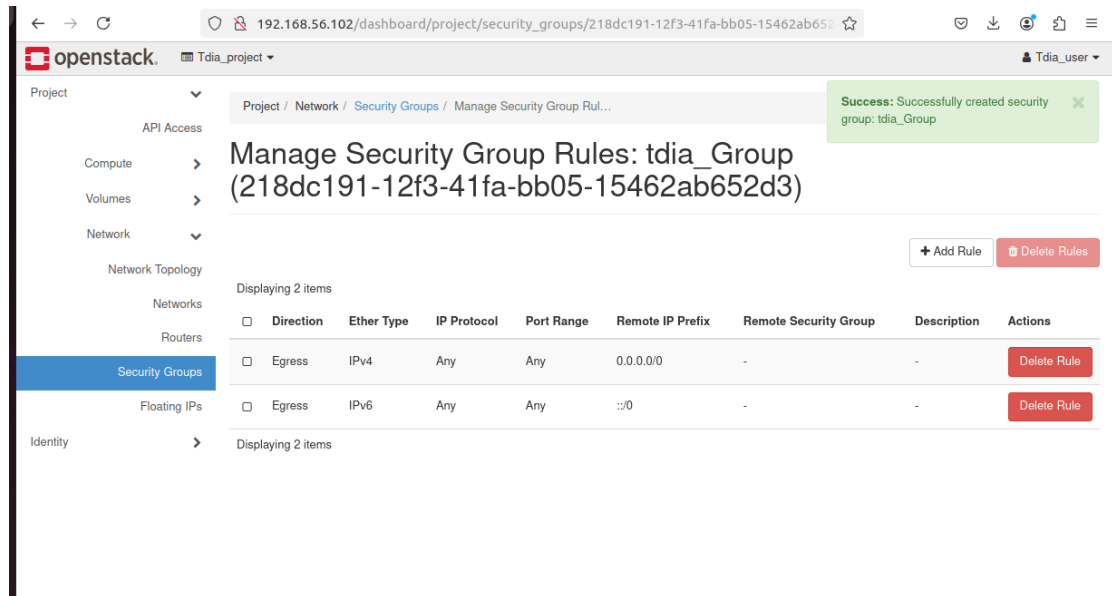
In OpenStack, a security group is a set of rules that control the incoming and outgoing network traffic for instances.

A) Navigate to “Project” , “Network” and then “Security Group”:



B) Define the Group Name: “tdia\_\_Group” :





### C) Add Rules to the Security Group:

by default in OpenStack, a security group allows all outbound traffic from a VM (Virtual Machine). This means that, unless specified otherwise, any traffic leaving the VM to the external network is permitted, regardless of the destination or protocol.

#### Outbound Traffic (Egress):

By default, all outgoing traffic from the VM is allowed. You don't need to define specific rules for outgoing traffic unless you want to restrict it.

#### Inbound Traffic (Ingress):

For incoming traffic, OpenStack security groups are more restrictive **by default**. Typically, only specific ports (such as SSH on port 22 or HTTP on port 80) are allowed, depending on the security group rules you define.

Specify inbound and outbound rules to allow or block specific types of traffic.

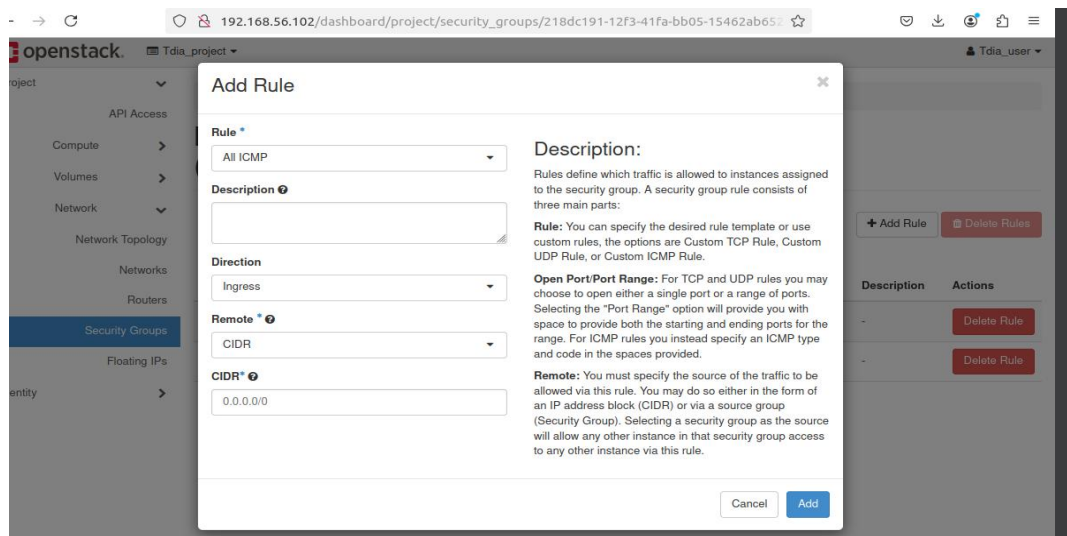
For example:

- Allow SSH (port 22) for remote access.
- Allow HTTP (port 80) and HTTPS (port 443) for web servers.
- Allow ICMP (ping) for diagnostics.

### A) Add rule : ALL ICMP

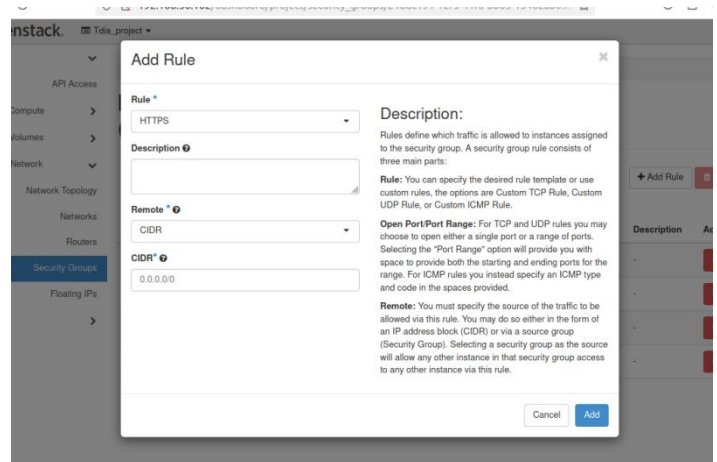
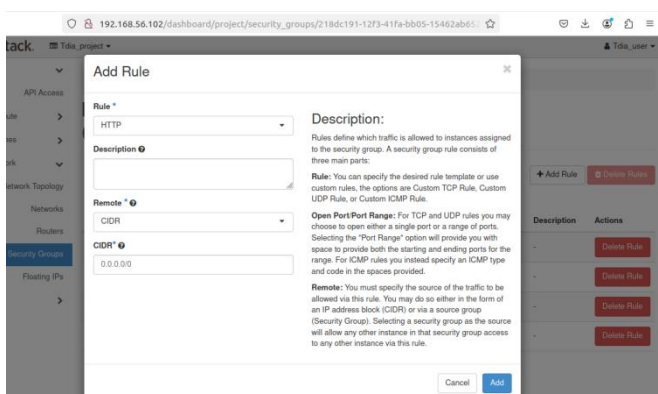
To add a rule that allows all ICMP traffic (which includes ping requests and replies), you need to configure the security group in OpenStack. ICMP traffic is used for diagnostic purposes, like using the ping command to check network connectivity.

This can be useful for network diagnostics but should be used carefully in production environments as **it can expose your VMs to unnecessary network traffic.**



## B) Add rule : HTTP and HTTPS

These rules are commonly used to allow web servers to communicate with users over the internet.



## C) Add rule : SSH

which is very important for us to be able to administer our machine later on.

**Add Rule**

**Rule \***  
SSH

**Description**

**Remote \***  
CIDR

**CIDR \***  
0.0.0.0/0

**Description:**  
Rules define which traffic is allowed to instances assigned to the security group. A security group rule consists of three main parts:  
**Rule:** You can specify the desired rule template or use custom rules, the options are Custom TCP Rule, Custom UDP Rule, or Custom ICMP Rule.  
**Open Port/Port Range:** For TCP and UDP rules you may choose to open either a single port or a range of ports. Selecting the "Port Range" option will provide you with space to provide both the starting and ending ports for the range. For ICMP rules you instead specify an ICMP type and code in the spaces provided.  
**Remote:** You must specify the source of the traffic to be allowed via this rule. You may do so either in the form of an IP address block (CIDR) or via a source group (Security Group). Selecting a security group as the source will allow any other instance in that security group access to any other instance via this rule.

Cancel Add

The set of rules created

Project / Network / Security Groups / Manage Security Group Rules...

### Manage Security Group Rules: tdia\_Group (218dc191-12f3-41fa-bb05-15462ab652d3)

+ Add Rule Delete Rules

Displaying 6 items

| <input type="checkbox"/> | Direction | Ether Type | IP Protocol | Port Range  | Remote IP Prefix | Remote Security Group | Description | Actions     |
|--------------------------|-----------|------------|-------------|-------------|------------------|-----------------------|-------------|-------------|
| <input type="checkbox"/> | Egress    | IPv4       | Any         | Any         | 0.0.0.0/0        | -                     | -           | Delete Rule |
| <input type="checkbox"/> | Egress    | IPv6       | Any         | Any         | :::0             | -                     | -           | Delete Rule |
| <input type="checkbox"/> | Ingress   | IPv4       | ICMP        | Any         | 0.0.0.0/0        | -                     | -           | Delete Rule |
| <input type="checkbox"/> | Ingress   | IPv4       | TCP         | 22 (SSH)    | 0.0.0.0/0        | -                     | -           | Delete Rule |
| <input type="checkbox"/> | Ingress   | IPv4       | TCP         | 80 (HTTP)   | 0.0.0.0/0        | -                     | -           | Delete Rule |
| <input type="checkbox"/> | Ingress   | IPv4       | TCP         | 443 (HTTPS) | 0.0.0.0/0        | -                     | -           | Delete Rule |

Displaying 6 items

## Step 7: Creating a New Flavor in OpenStack

Creating a flavor in OpenStack allows you to define the specifications (vCPUs, RAM, disk, etc.) for virtual machines (VMs) that users can launch.

Then we connect again as admin:



openstack®

Log in

User Name

Password

Sign In

**Creating a new flavor** by specifying the resources we want:

Name: m1.test

VCPUs: 1

RAM: 1024 MB

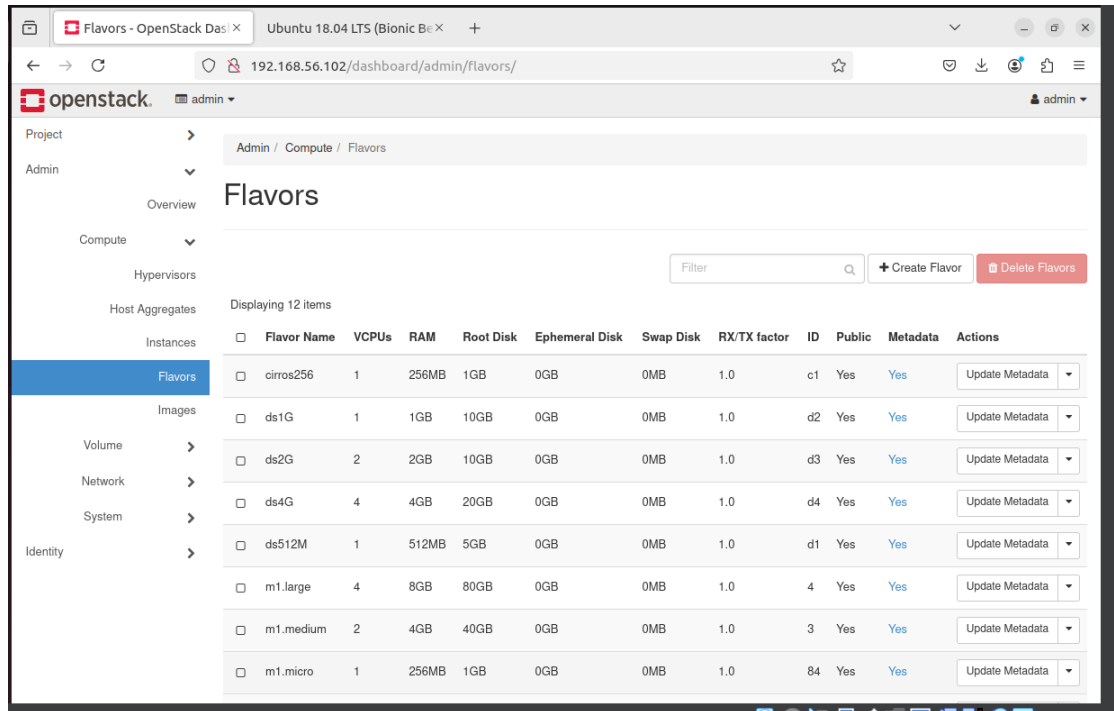
ROOT DISK: 10 GB

(while respecting the characteristics we specified during the creation of our image)

Navigate to flavor Section :

Click on the "Admin" tab (or "System", depending on your OpenStack version).

From the side menu, select "Flavors" under the "Compute" category.



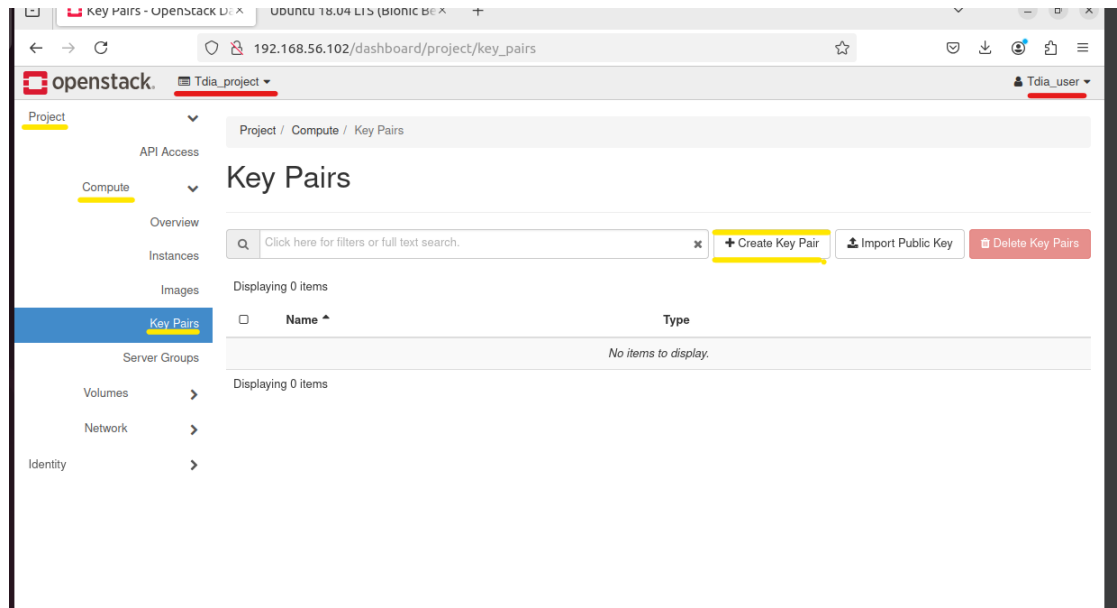
## Fill in the New Flavor Details:

- **Name:** Provide a unique name for the flavor (e.g., m1.test).
- **ID:** You can assign a specific ID or leave it blank for the system to auto-generate one.
- **vCPUs:** Enter the number of virtual CPUs for this flavor (e.g., 1).
- **RAM (MB):** Specify the amount of RAM in megabytes (e.g., 1024 for 1 GB).
- **Root Disk (GB):** Enter the size of the root disk in gigabytes (e.g., 10).
- **Ephemeral Disk (GB):** Specify the size of the ephemeral disk, if any (optional).
- **Swap Disk (MB):** Enter the swap disk size in megabytes (optional).
- **RX/TX Factor:** This is used for bandwidth scaling (leave as 1.0 if unsure).

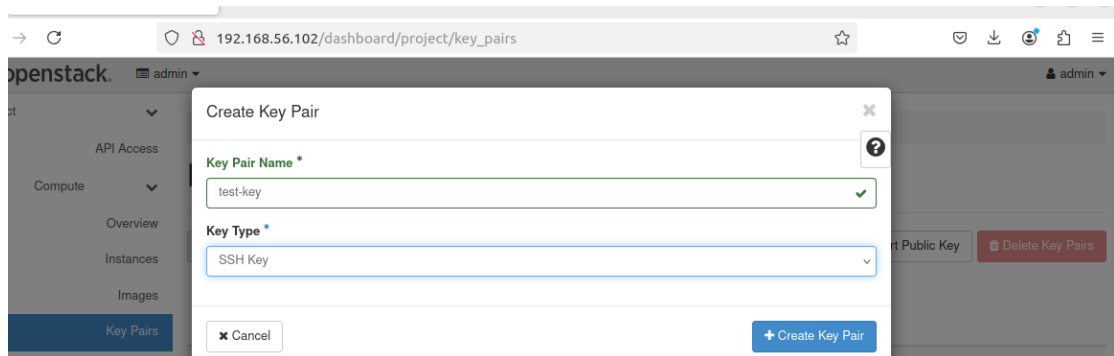
## Etape 8 : création d'une paire de clé



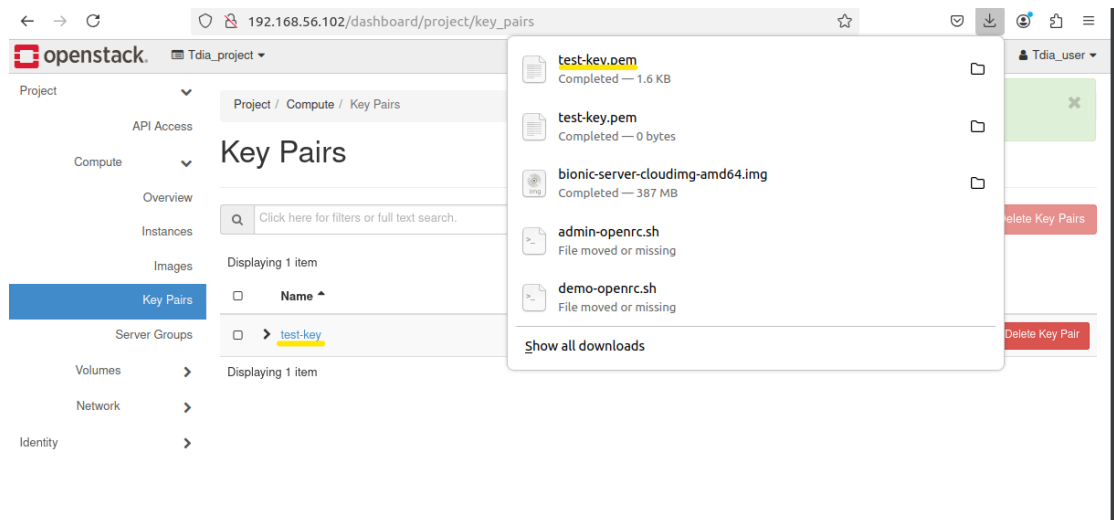
Connect as Tdia\_user and navigate to “key Pairs” section



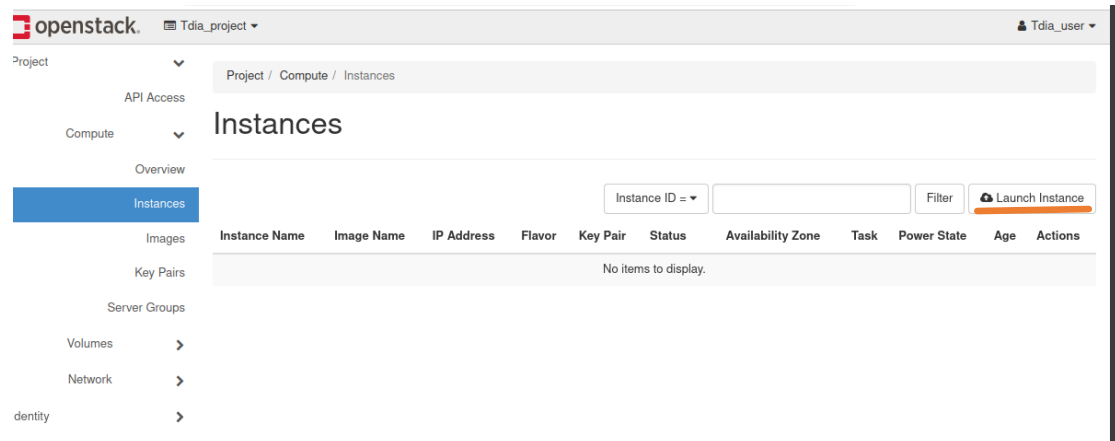
The SSH key test-key that we are asked to create, then we retrieve the generated key to use during SSH connection



As we can see, the key is generated automatically and downloaded to your machine.

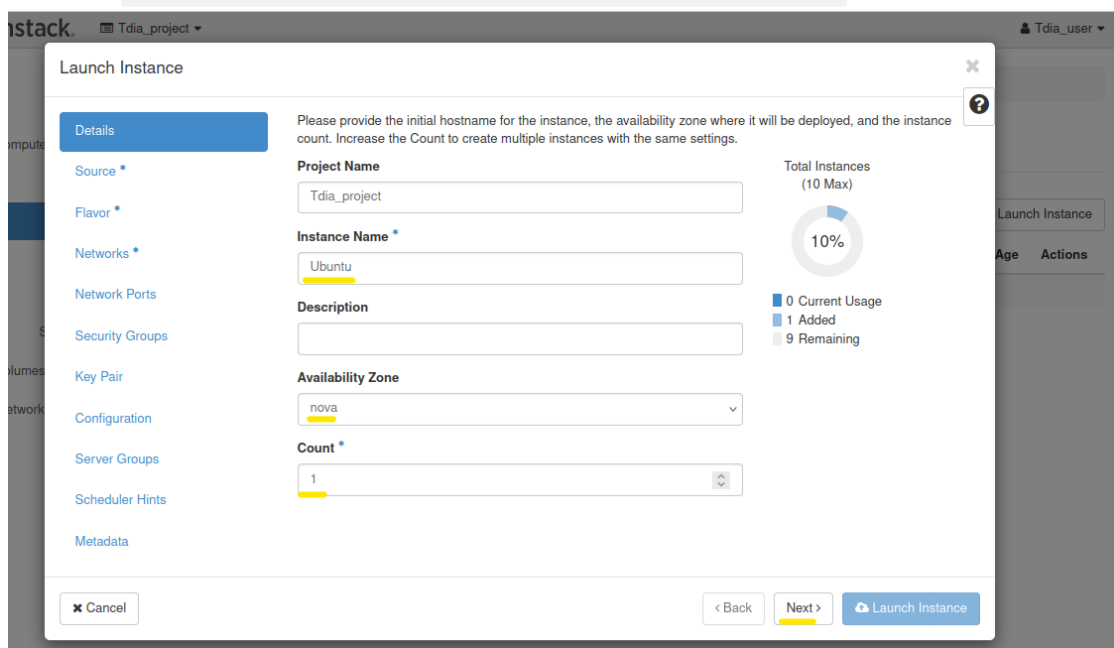


## Step 9: Creating the instance



Set Instance Details [Project Name : The scope of your Instance,  
Name : Ubuntu ,

Description : (optional)]



The 'Launch Instance' dialog box is shown with the 'Details' tab selected. It contains fields for Project Name, Instance Name, Description, Availability Zone, and Count. A progress indicator shows 10% completion. The 'Launch Instance' button is highlighted.

**Launch Instance**

Please provide the initial hostname for the instance, the availability zone where it will be deployed, and the instance count. Increase the Count to create multiple instances with the same settings.

**Details**

Source \*  
Flavor \*  
Networks \*  
Network Ports  
Security Groups  
Key Pair  
Configuration  
Server Groups  
Scheduler Hints  
Metadata

**Project Name**  
Tdia\_project

**Instance Name \***  
Ubuntu

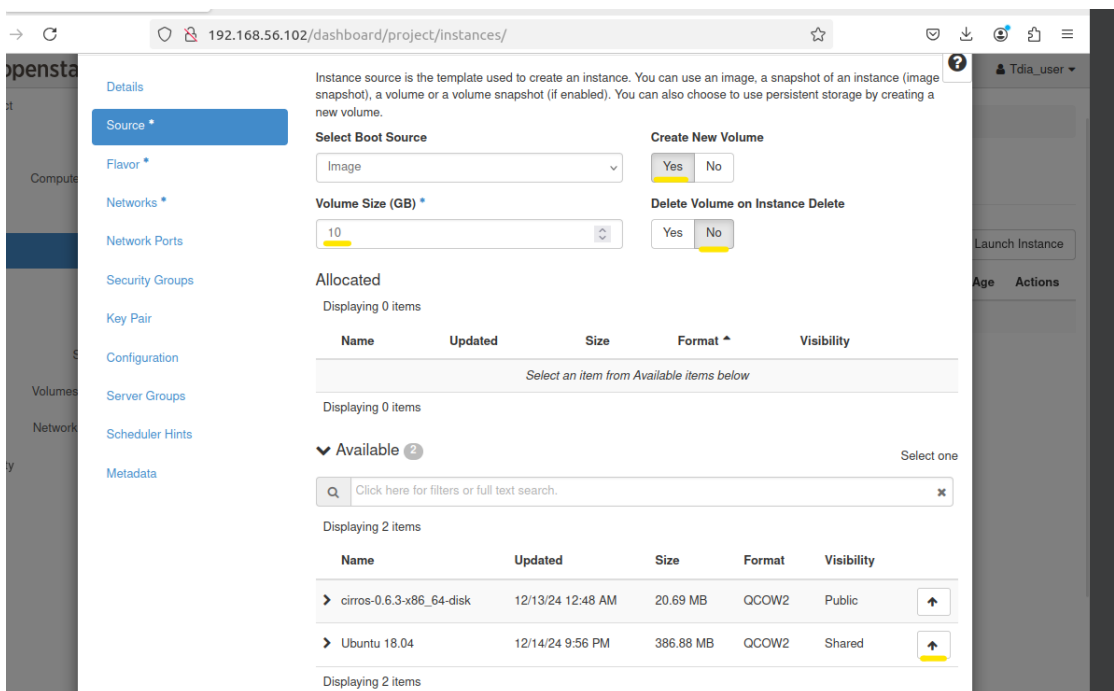
**Description**

**Availability Zone**  
nova

**Count \***  
1

**Total Instances (10 Max)**  
10%  
0 Current Usage  
1 Added  
9 Remaining

**Buttons:** Cancel, Back, Next, Launch Instance



The 'Instance Source' configuration page is shown. It includes sections for 'Select Boot Source', 'Create New Volume', 'Delete Volume on Instance Delete', 'Allocated' items, and 'Available' items. The 'Available' section lists two items: 'cirros-0.6.3-x86\_64-disk' and 'Ubuntu 18.04'.

**Instance Source**

Instance source is the template used to create an instance. You can use an image, a snapshot of an instance (image snapshot), a volume or a volume snapshot (if enabled). You can also choose to use persistent storage by creating a new volume.

**Select Boot Source**  
Image

**Create New Volume**  
Yes No

**Volume Size (GB) \***  
10

**Delete Volume on Instance Delete**  
Yes No

**Allocated**  
Displaying 0 items

| Name                                      | Updated | Size | Format | Visibility |
|-------------------------------------------|---------|------|--------|------------|
| Select an item from Available items below |         |      |        |            |

Displaying 0 items

**Available** 2

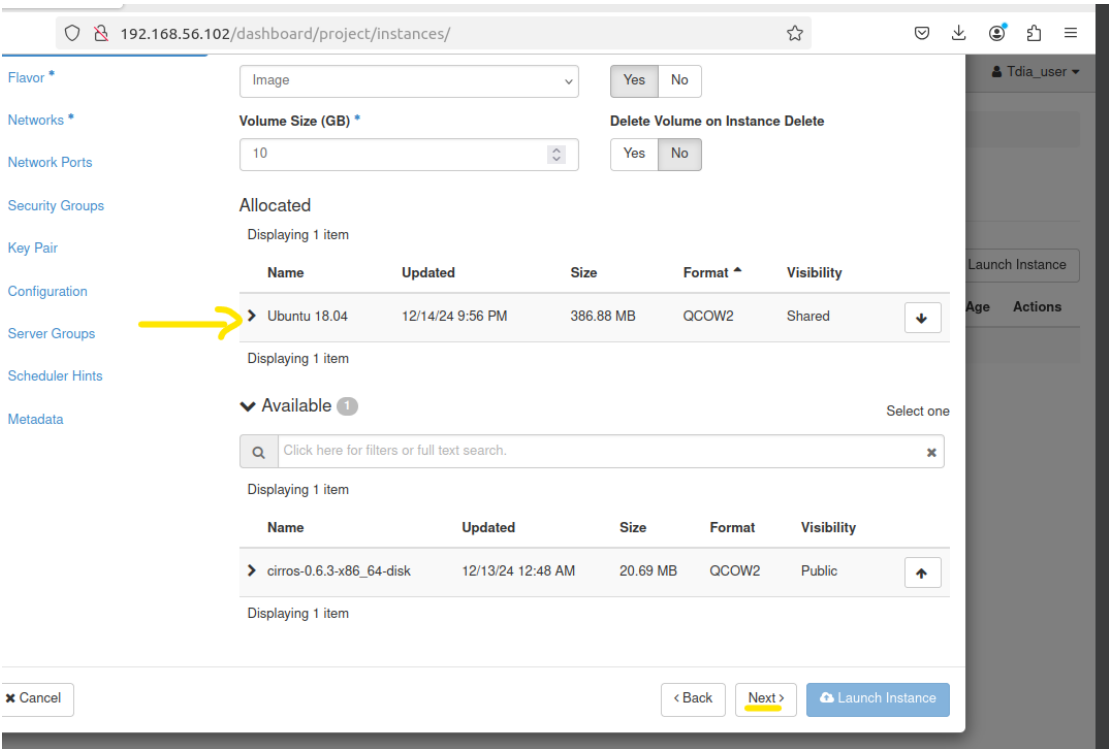
Click here for filters or full text search.

Displaying 2 items

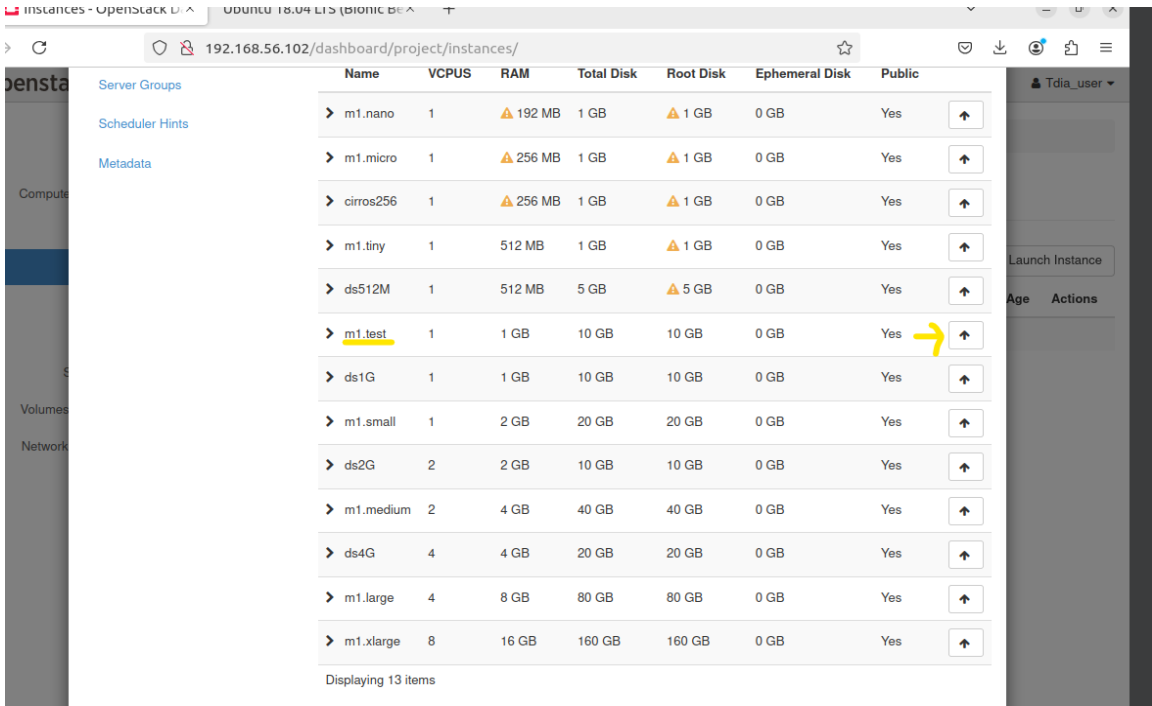
| Name                       | Updated           | Size      | Format | Visibility |
|----------------------------|-------------------|-----------|--------|------------|
| > cirros-0.6.3-x86_64-disk | 12/13/24 12:48 AM | 20.69 MB  | QCOW2  | Public     |
| > Ubuntu 18.04             | 12/14/24 9:56 PM  | 386.88 MB | QCOW2  | Shared     |

Displaying 2 items

Click on the up-arrow associated with the Ubuntu image we downloaded earlier.



Then select the flavor we created for this image: m1.test



Then we select the private network we created: test-private

Launch Instance

Details  
Source  
Flavor  
**Networks**  
Network Ports  
Security Groups  
Key Pair  
Configuration  
Server Groups  
Scheduler Hints  
Metadata

Networks provide the communication channels for instances in the cloud. You can select ports instead of networks or a mix of both.

▼ Allocated 1  
Displaying 1 item

| Network        | Subnets Associated  | Shared | Admin State | Status |
|----------------|---------------------|--------|-------------|--------|
| > test-private | test-private-subnet | No     | Up          | Active |

Displaying 1 item

▼ Available 1  
Select one or more

Click here for filters or full text search.

Displaying 1 item

| Network  | Subnets Associated | Shared | Admin State | Status |
|----------|--------------------|--------|-------------|--------|
| > shared | shared-subnet      | No     | Up          | Active |

Displaying 1 item

Cancel < Back Next > Launch Instance

(#!Skip **Network Post** part )

Add the group we created before :

Launch Instance

Details  
Source  
Flavor  
Networks  
Network Ports  
**Security Groups**  
Key Pair  
Configuration  
Server Groups  
Scheduler Hints  
Metadata

Select the security groups to launch the instance in.

▼ Allocated 1  
Displaying 1 item

| Name         | Description |
|--------------|-------------|
| > tdia_Group |             |

Displaying 1 item

▼ Available 1  
Select one or more

Click here for filters or full text search.

Displaying 1 item

| Name      | Description            |
|-----------|------------------------|
| > default | Default security group |

Displaying 1 item

Cancel < Back Next > Launch Instance

Select the Key 'key-test' :

Launch Instance

Details

Source

Flavor

Networks

Network Ports

Security Groups

**Key Pair**

Configuration

Server Groups

Scheduler Hints

Metadata

A key pair allows you to SSH into your newly created instance. You may select an existing key pair, import a key pair, or generate a new key pair.

[+ Create Key Pair](#) [Import Key Pair](#)

**Allocated**

Displaying 1 item

| Name       | Type |
|------------|------|
| > test-key | ssh  |

Displaying 1 item

**Available** 0

Select one

Displaying 0 items

| Name                 | Type |
|----------------------|------|
| No items to display. |      |

Displaying 0 items

[Cancel](#) [Back](#) [Next >](#) [Launch Instance](#)

And then skip all other steps (keep them in default values) and lunch the instance.

openstack.

Test\_project

Test\_user

Project

API Access

Compute

Overview

Instances

Images

Key Pairs

Server Groups

Volumes

Network

Identity

Project / Compute / Instances

Instances

Instance ID = 

Filter

Launch Instance

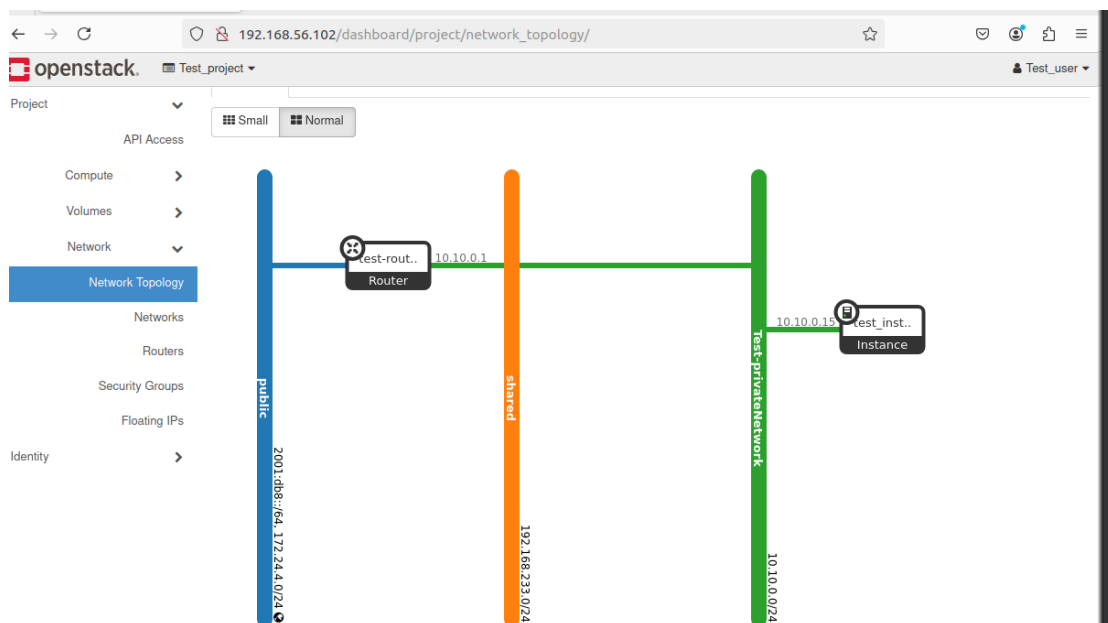
Delete Instances

More Actions

Displaying 1 item

| <input type="checkbox"/> | Instance Name                 | Image Name | IP Address | Flavor                  | Key Pair | Status | Availability Zone | Task | Power State | Age     | Actions   |                            |
|--------------------------|-------------------------------|------------|------------|-------------------------|----------|--------|-------------------|------|-------------|---------|-----------|----------------------------|
| <input type="checkbox"/> | <a href="#">test_instance</a> | -          | 10.10.0.15 | <a href="#">m1.test</a> | test_key | Active | ml                | nova | None        | Running | 3 minutes | <div>Create Snapshot</div> |

Displaying 1 item



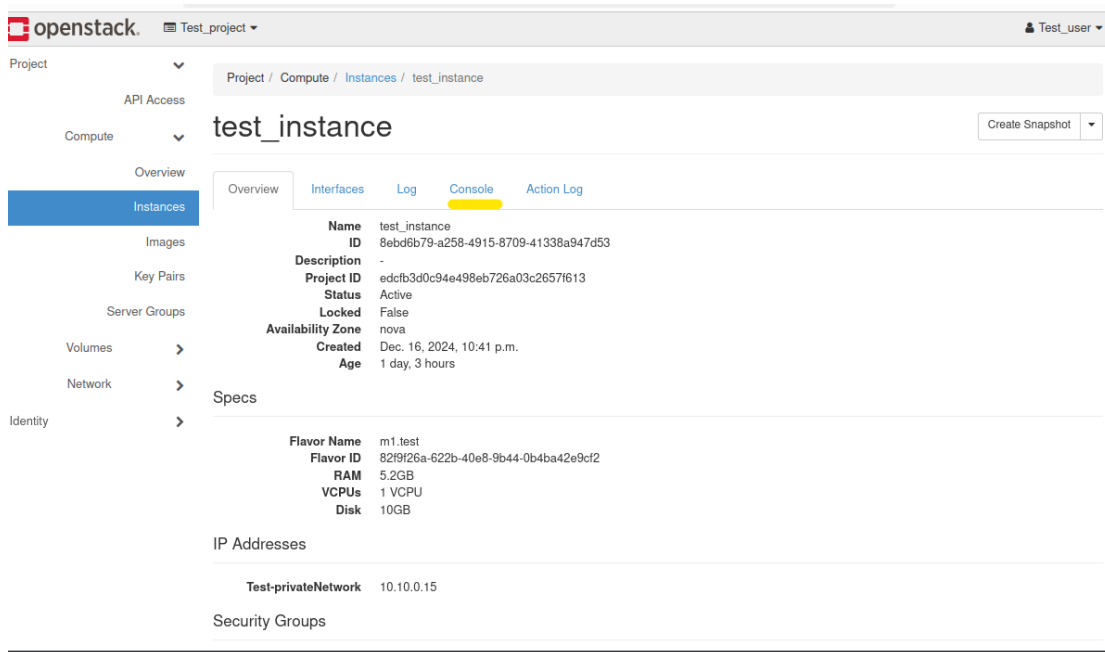
Our network topology after creating the instance.

### ➤ Step8: Launching the instance:

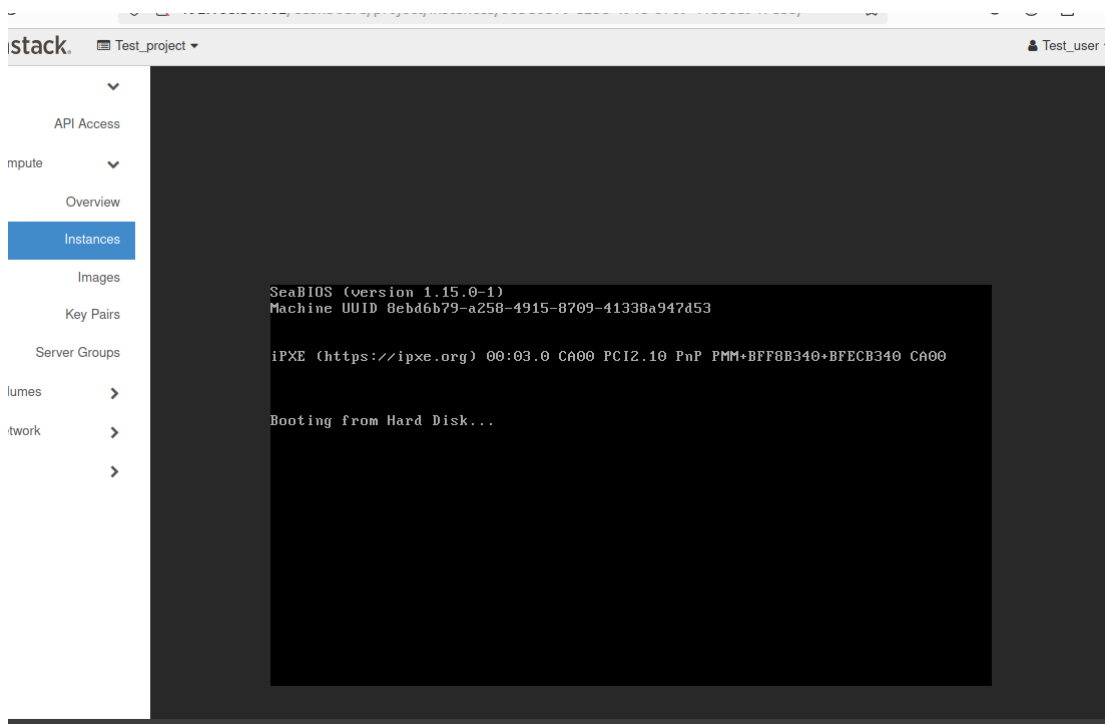
The screenshot shows the 'Instances' page in the OpenStack dashboard. The page title is 'Instances' and it shows a table with one instance. The instance is named 'test\_instance' and is in a 'Shutoff' state. The table columns include Instance Name, Image Name, IP Address, Flavor, Key Pair, Status, Availability Zone, Task, Power State, Age, and Actions. The 'Start Instance' button is visible in the Actions column for the 'test\_instance' row.

| Instance Name | Image Name   | IP Address | Flavor  | Key Pair | Status  | Availability Zone | Task | Power State | Age            | Actions        |
|---------------|--------------|------------|---------|----------|---------|-------------------|------|-------------|----------------|----------------|
| test_instance | Ubuntu-te st | 10.10.0.15 | m1.test | test_key | Shutoff | nova              | None | Shut Down   | 1 day, 3 hours | Start Instance |

If your instance is stopped, click “Start Instance” and then click the instance name.



Click on “Console” to interact with your instance via command line.



### ➤ Step10: Creating a CirrOS Instance:

Since launching an Ubuntu machine requires significant resources, we chose to launch a minimalist CirrOS machine provided by OpenStack to test instance deployment:

**CirrOS** is a lightweight virtual machine image specifically designed for use in cloud computing environments, including OpenStack.



- **Lightweight Nature:** CirrOS is an extremely lightweight and minimal Linux distribution, optimized for cloud computing environments. Its small size makes it ideal for quick deployments and agile testing.
- **Fast Boot:** The CirrOS image is designed for rapid booting, making it an efficient choice for scenarios where deployment speed is critical, such as automated testing and cloud experiments.

We chose the name: **cirros**.

# VII. Challenges encountered

---

## **1. Network Configuration:**

Network configuration is a crucial aspect when setting up a private cloud with OpenStack. Below is a detailed explanation of the potential challenges we encountered while working with Ubuntu virtual machines and OpenStack. Configuring the network in an OpenStack environment can be complex, especially if you have specific requirements for security, redundancy, or performance:

### **Issues with IP Address Configuration:**

- **Incorrect Configuration:** Misconfigured IP addresses on the virtual machine or at the OpenStack level.

Possible Solutions:

- **Check IP Configuration:** Ensure that the IP addresses for the virtual machine are correctly set up. Also, verify that the OpenStack subnets are configured properly.

### **Routing Issues Between Private and Public Networks:**

- **Routing Problems:** Incorrect routing between private and public networks can lead to connectivity loss.

Possible Solutions:

- **Configure Routing:** Ensure that the routing between private and public networks is set up correctly. Check the routing tables in OpenStack.

### **SSH Connection Issues:**

- **Firewall:** Firewall rules may block SSH connections.

Possible Solutions:

- **Configure Firewall Rules:** Open the SSH port (typically port 22) in the virtual machine's firewall and ensure that OpenStack's security rules allow SSH traffic.

## **Resource Management:**

Resource management is a critical aspect of setting up a private cloud, whether on a physical or virtual machine. Below are some specific challenges you may encounter, particularly when setting up a private cloud with an Ubuntu virtual machine:

### **RAM Allocation and Management:**

- **Virtual Machine Configuration:** The initial configuration of the virtual machine, such as allocating RAM, virtual CPUs, and disk size, can significantly impact its resource needs and lead to instance launch issues under OpenStack.
- **Overcommitment:** Poor planning may result in RAM overcommitment.  
Possible Solutions:
  - **Monitor RAM Usage:** Use tools like `top` or `htop` to monitor RAM usage on the Ubuntu virtual machine.
  - **Adjust Allocation:** If necessary, adjust the RAM allocation in the virtual machine settings via OpenStack.

### **Storage Management:**

- **Storage Shortage:** Storage can become an issue if virtual machines generate large amounts of data or if VM images take up too much space.  
Possible Solutions:
  - **Monitor Storage Usage:** Use commands to monitor storage usage on the virtual machine.
  - **Resize Storage:** If necessary, resize the storage volumes associated with the virtual machine via OpenStack.

## **Disk Performance:**

- **Poor Disk Performance:** Virtual disk performance can be lower than physical disk performance, especially when using shared storage.  
Possible Solutions:
  - **Use Fast Storage:** Where possible, use faster storage solutions to enhance performance.
  - **Optimize I/O:** Opt for disk configurations that are optimized for specific workloads.

# Conclusion

---

**The objective of our project** is to set up a private cloud using OpenStack. The first step was to provide a general overview of cloud computing, its architecture, and its various services. Subsequently, we conducted a study of different popular private cloud solutions, highlighting the virtualization techniques used in each of them.

The study enabled us to select the most suitable solution: **OpenStack**.

OpenStack has an open-source cloud computing architecture that offers a robust and scalable cloud infrastructure. Its modular architecture consists of several interconnected components, providing significant flexibility and extensibility. The key modules of OpenStack include **Nova** for virtual machine management, **Neutron** for network management, **Cinder** for block storage, **Glance** for image management, and **Keystone** for identity management. This architecture facilitates the creation of private clouds tailored to a variety of workloads. OpenStack provides capabilities such as horizontal scalability, automated resource management, high availability, and a user-friendly interface. It supports multiple hypervisors. With its advanced features, OpenStack plays a critical role in building and managing flexible and efficient cloud infrastructures.

In the area of **network configuration**, we overcame challenges such as IP address configuration and managing private and public networks. By adopting a methodical approach and following OpenStack's best practices, we were able to establish a robust network infrastructure.

**Resource management** was another major concern, particularly regarding RAM allocation, storage management, and disk performance optimization.

This project also allowed us to develop valuable skills in problem-solving, team collaboration, and adaptability to technological challenges. The knowledge gained on OpenStack, network configuration, and resource management has strengthened our expertise in the field of cloud infrastructures.

# Bibliography

---

<https://docs.openstack.org/sahara/pike/intro/architecture.html>  
<https://docs.openstack.org/horizon/queens/user/manage-volumes.html>  
<https://docs.openstack.org/2024.2/install/>  
<https://anuruddhall.blogspot.com/2015/03/creating-instance-in-openstack.html>  
<https://adri-v.medium.com/a-beginners-guide-to-openstack-9bc024f4c4e3>  
<https://medium.com/@hdhakad54/how-to-install-openstack-on-ubuntu-20-04-with-devstack-678549fbf9cb>  
<https://www.youtube.com/watch?v=D6J4wEn-R0Y&t=214s>  
<https://www.youtube.com/watch?v=D2zMc5Fw4m0> [https://medium.com/@kcoupal/how-to-install-openstack-on-ubuntu-22-04-with-devstack- 3336c01ddcfa](https://medium.com/@kcoupal/how-to-install-openstack-on-ubuntu-22-04-with-devstack-3336c01ddcfa)